# QUANTGEN Models Primer

The price-path models behind the Quant Lab: log returns, Monte Carlo, GBM, GARCH(1,1) with Student-t innovations, maximum likelihood, its guardrails and its precision, and the drift study.

Built 2026-09-27 10:56 by `docs/build_models_primer.py` from the code in `core/quant/`.

Data behind every fitted number and figure 1-6: AAPL daily closes from FMP, 504 sessions, 2024-09-20 to 2026-09-24.

Written for the operator of QUANTGEN. Nothing in this document is investment advice; it explains what the software computes and why.

# Contents

**0. How to use this document . . . . . 4**

Where each part of the screen is explained . . . . . 4

How to read the numbers in this document . . . . . 5

**1. What a price-path model is . . . . . 6**

1.1 A model of tomorrow's return . . . . . 6

1.2 Daily log returns and why logs . . . . . 6

1.3 Annualising: 252 and the square root of 252 . . . . . 6

1.4 Two measures: what you simulate with and what you price with . . . . . 7

**2. Monte Carlo . . . . . 7**

2.1 A method, not a model . . . . . 7

2.2 Seeds and reproducibility . . . . . 7

2.3 The percentile fan . . . . . 7

2.4 VaR and CVaR from the terminal distribution . . . . . 8

2.5 Probability of loss and of reaching a target . . . . . 9

2.6 The standard error of a Monte Carlo number . . . . . 9

2.7 What changes with the horizon . . . . . 10

**3. Geometric Brownian motion (GBM) . . . . . 10**

3.1 The equation and its discrete step . . . . . 10

3.2 Calibration and the $-\frac{1}{2}\sigma^2$ term . . . . . 10

3.3 What GBM gets right and wrong . . . . . 11

3.4 Jump diffusion: patching the tails . . . . . 11

3.5 Block bootstrap: no distribution at all . . . . . 11

3.6 The four models side by side . . . . . 11

**4. Volatility clustering and fat tails . . . . . 12**

4.1 Clustering: large moves follow large moves . . . . . 12

4.2 Fat tails: excess kurtosis . . . . . 12

4.3 Why a constant-vol fan is wrong in both directions . . . . . 13

**5. GARCH(1,1) . . . . . 13**

5.1 The equation, term by term . . . . . 13

5.2 Persistence and the unconditional variance . . . . . 14

5.3 The half-life of a variance shock . . . . . 14

5.4 The conditional volatility path . . . . . 14

5.5 Forecasting the variance h sessions ahead . . . . . 15

5.6 IGARCH and the near-integrated flag . . . . . 16

5.7 Typical values and what unusual ones mean . . . . . 16

5.8 How precisely are α, β and the persistence known? . . . 17
**6. Student-t innovations . . . 18**
6.1 The t distribution and ν . . . 18
6.2 Scaling to unit variance: the classic bug . . . 18
6.3 Reading ν, and the η = 1/ν parameterisation . . . 19
6.4 The likelihood-ratio test: t versus normal, with the null on the boundary . . . 19
6.5 How precisely is ν known? The profile interval . . . 20
**7. Maximum likelihood, log-likelihood, and the guardrails . . . 21**
7.1 What a likelihood is . . . 21
7.2 Why the log . . . 22
7.3 The per-observation formulas, as implemented . . . 22
7.4 AIC and BIC . . . 22
7.5 Numerical optimisation: L-BFGS-B, starts, bounds, convergence . . . 23
7.6 The guardrails, one by one . . . 23
7.7 The modifications ladder . . . 26
7.8 How to tell that something broke . . . 28
**8. Drift . . . 28**
8.1 The least identifiable input . . . 28
8.2 The estimators compared . . . 29
8.3 The protocol . . . 29
8.4 Results . . . 29
8.5 Reading the results . . . 35
8.6 What the app offers now, and the default . . . 36
**9. Reading the Quant Lab screen . . . 37**
9.1 The first stat strip . . . 37
9.2 The chips . . . 38
9.3 The GARCH strip and the guardrail chips . . . 38
9.4 The three charts . . . 39
9.5 The narrative and every sentence it can produce . . . 39
**10. Glossary and formula sheet . . . 41**
10.1 Symbols and terms . . . 41
10.2 Formula sheet . . . 41
10.3 The worked fit in one line . . . 42
10.4 References . . . 42

# 0. How to use this document

The Quant Lab takes a symbol, a data source, a model, a drift mode, a horizon in sessions and a number of paths. It loads up to 504 daily closes, converts them to daily log returns, calibrates the model on those returns, draws the paths from a fixed seed, and answers with a stat strip, a row of chips, three charts and a written narrative. When the model is GARCH(1,1)-t (the default) a second stat strip shows the fitted parameters and one chip per guardrail that the fit hit. This document explains every one of those numbers, in the order you need them, using values computed from the same code when the PDF was built.

## Where each part of the screen is explained

| Screen element | Chapter |
|---|---|
| Symbol, data source, horizon, paths, target | 1 (returns), 2 (Monte Carlo) |
| Model dropdown: GBM, jump diffusion, block bootstrap | 3 |
| Model dropdown: GARCH(1,1)-t | 4 (why), 5 (the model), 6 (the tails) |
| Drift dropdown and the ERP field | 8 (the study), 1.4 (which measure) |
| Spot, median terminal, 90% range, P(loss), VaR 95, CVaR, P(target) | 2 |
| Drift (ann.), Vol (ann.) tiles; the drift sentence; the rf chip | 3, 8 |
| alpha, beta, alpha+beta, half-life, nu, cond / uncond / horizon vol | 5, 6 |
| The intervals in brackets on those tiles: "value (low to high)" | 5.8 (alpha, beta, persistence, half-life), 6.5 (nu) |
| LR t vs normal, dAIC vs GBM | 6.4, 7.4 |
| Chips: at bound, solver did not converge, SE undefined, interval touches a bound, guardrails clear, normal innovations, persistence interval reaches 1 | 7.6 |
| Conditional vol chart, fan chart, terminal histogram | 5.4, 2.3, 2.4 |
| The narrative panel and every sentence it can produce | 9 |

**Safe to change / do not change without reading chapter 7**

- **Safe, any time:** symbol, data source, horizon, number of paths, target price, drift mode, the ERP percentage, and the seed if you edit the call. They change the question, not the estimator.
- **Think first (chapter 7.7):** the nu level above which the t is treated as normal, the persistence cap, the alpha cap, the minimum-observation rule, the optimiser's starting values, the LR critical value, the profile-interval cuts. Each is a guardrail; moving one changes when the app warns you.
- **Do not touch without a test in hand (chapter 7.8):** the likelihood formulas, the unit-variance scaling of the t draws, the variance recursion and its starting value, the omega scaling, the annualisation constant. A mistake there corrupts every number silently: the screen still fills with plausible values.

## How to read the numbers in this document

Every fitted or simulated number was produced when this file was built, by calling `fit_garch`, `run_monte_carlo` and their helpers on: AAPL daily closes from FMP, 504 sessions, 2024-09-20 to 2026-09-24. The drift-study tables are read from `docs/drift_study/results.csv`. To rebuild after a code change, run `python docs/build_models_primer.py` from the project root; it needs no network (set PRIMER_OFFLINE=1 to force the synthetic fallback) and finishes in well under two minutes.

One series appears throughout as the worked example, so you can follow a single fit from raw returns to the fan chart. A second series, the app's own no-key demo (`synthetic_history("AAPL")`, a geometric Brownian path), is used wherever a guardrail needs to be shown binding: it is exactly what the Quant Lab shows when you choose the "Synthetic demo" data source.

# 1. What a price-path model is

## 1.1 A model of tomorrow's return

A price-path model is a statement about the probability distribution of the next daily return. Every model in the Quant Lab, however elaborate, can be written as

$$r_{t+1} = \mu + \sigma_{t+1} z_{t+1}, \qquad \mathrm{E}[z] = 0, \qquad \mathrm{Var}[z] = 1$$

with three ingredients. The **drift** μ is the expected daily return. The **volatility** $\sigma_{t+1}$ is the standard deviation of the return, which may be constant (GBM) or change from day to day (GARCH). The **innovation** z is a random draw with mean zero and variance one whose shape decides how likely extreme days are: normal for GBM, Student-t for GARCH-t, the sample's own history for the bootstrap. Choosing a model means choosing those three things. Everything the lab reports — the fan, VaR, the probability of loss — follows from that choice plus arithmetic.

## 1.2 Daily log returns and why logs

The app never models prices directly. It converts the close series $P_t$ into daily log returns

$$r_t = \ln(P_t / P_{t-1})$$

(`History.log_returns()` is `np.diff(np.log(px))`). Logs are used for two reasons. First, they add: the log return over a week is the sum of the five daily log returns, so a horizon of h sessions is just a sum of h draws, and sums of independent draws are what statistics knows how to handle. Simple returns compound multiplicatively and do not add. Second, exponentiating a sum of log returns can never give a negative price: the simulated price is $P_0 \exp(r_1 + \ldots + r_h)$. For small moves the two are almost equal (a 1% simple return is a 0.995% log return); for large moves they differ, and the app's percentages are always converted back to prices before they are shown.

## 1.3 Annualising: 252 and the square root of 252

There are about 252 trading sessions in a year, so a daily mean becomes an annual mean by multiplying by 252. Variances of independent daily returns also add, so the annual variance is 252 times the daily variance, and the annual *volatility* (its square root) is √252 ≈ 15.87 times the daily volatility:

$$\mu_{\text{annual}} = 252\,\mu_{\text{daily}}, \qquad \sigma_{\text{annual}} = \sqrt{252}\,\sigma_{\text{daily}}$$

On the worked series (T = 503 daily returns) the sample mean is +0.0769% per day, i.e. +19.4% a year, and the sample standard deviation is 1.802% per day, i.e. 28.6% a year. Inside `core/quant/garch.py` everything is kept in daily units and annualised only at the edges (`TRADING_DAYS = 252`). The FICC pages quote rates in percent and changes in basis points; here volatility is quoted in percent per year and returns in percent.

### 1.4 Two measures: what you simulate with and what you price with

The Quant Lab simulates under the **physical measure**: the drift is meant to be the return you actually expect to earn, and the fan is a statement about what may happen to a position. That is the right object for risk: VaR, the probability of loss, the width of the cone. Its weakness is that the expected return is the least identifiable number in finance (chapter 8), so the physical measure is only as good as the drift you put in.

The pricing pages (options, structured notes, swaptions) work under the **risk-neutral measure**. There the drift of every asset is replaced by the risk-free rate — not because anyone expects a stock to earn the T-bill rate, but because a hedged position earns the risk-free rate, and a price that assumed anything else could be arbitraged. A risk-neutral simulation prices; it does not forecast. The Quant Lab's *rf* drift mode is that measure: it is offered so you can see what the pricing pages assume, and it is labelled as not-a-forecast for exactly this reason.

# 2. Monte Carlo

## 2.1 A method, not a model

Monte Carlo is a way of reading a distribution you cannot write down in closed form: draw many possible futures from the model, then count. The Quant Lab draws a matrix of daily log returns with one row per path and one column per session, cumulates each row, exponentiates, and multiplies by spot. The core of `run_monte_carlo` is four lines:

```
log_paths = np.cumsum(shocks, axis=1)                       # (n_paths, horizon)
paths = spot * np.exp(np.hstack([np.zeros((n_paths, 1)), log_paths]))
fan = {p: np.percentile(paths, p, axis=0) for p in FAN_PERCENTILES}  # 5,25,50,75,95
terminal = paths[:, -1].copy()
```

The model decides what goes into `shocks`; everything after that is the same for GBM, jump diffusion, the bootstrap and GARCH-t. That is why the risk numbers of the four models are directly comparable: they are read off the same kind of object with the same code.

## 2.2 Seeds and reproducibility

Every run starts from `np.random.default_rng(seed)` with seed 7 unless you pass another. The same inputs therefore draw exactly the same paths, and the acceptance test `test_mc_is_seeded_and_reproducible` checks that seed 7 twice gives the same median and seed 8 gives a different one. A simulation that repaints on every refresh cannot be reasoned about: you could never tell whether a change in VaR came from the market or from the dice.

## 2.3 The percentile fan

At each future session the app takes the 5th, 25th, 50th, 75th and 95th percentile of the simulated prices across paths. The shaded bands are those percentiles joined up over time; the median line is the 50th. None of them is a path: no single simulated future runs along the 95th percentile, so the fan says where the price may be on a given date, not how it gets there. The faint lines behind the fan are forty real paths, kept so the eye can see what one future looks like.

## 2.4 VaR and CVaR from the terminal distribution

Write the horizon loss as a fraction of spot, L = 1 − $P_T$ / spot (positive means a loss). **Value at risk** at level 95% is the loss that is exceeded on only 5% of paths; **conditional VaR** (expected shortfall) is the average loss on those 5% of paths:

$$\mathrm{VaR}_{95} = 95\text{th percentile of } L, \qquad \mathrm{CVaR}_{95} = \mathrm{E}[\, L \mid L \geq \mathrm{VaR}_{95} \,]$$

```
losses = 1.0 - terminal / spot                        # +ve = loss fraction
var = float(np.percentile(losses, var_level * 100))   # var_level = 0.95
tail = losses[losses >= var]
cvar = float(np.mean(tail)) if tail.size else var
```

Both are clipped at zero (a distribution so favourable that the 95th percentile loss is negative shows VaR 0.0%). On the worked series over 126 sessions with 5,000 paths, GARCH-t gives $VaR_{95}$ = 28.6% and CVaR = 37.6%; GBM on the same history and seed gives 27.7% and 33.2%. CVaR is always at least VaR (the test `test_mc_var_and_cvar_are_coherent` asserts it); the gap between them is a direct read of how fat the left tail is.

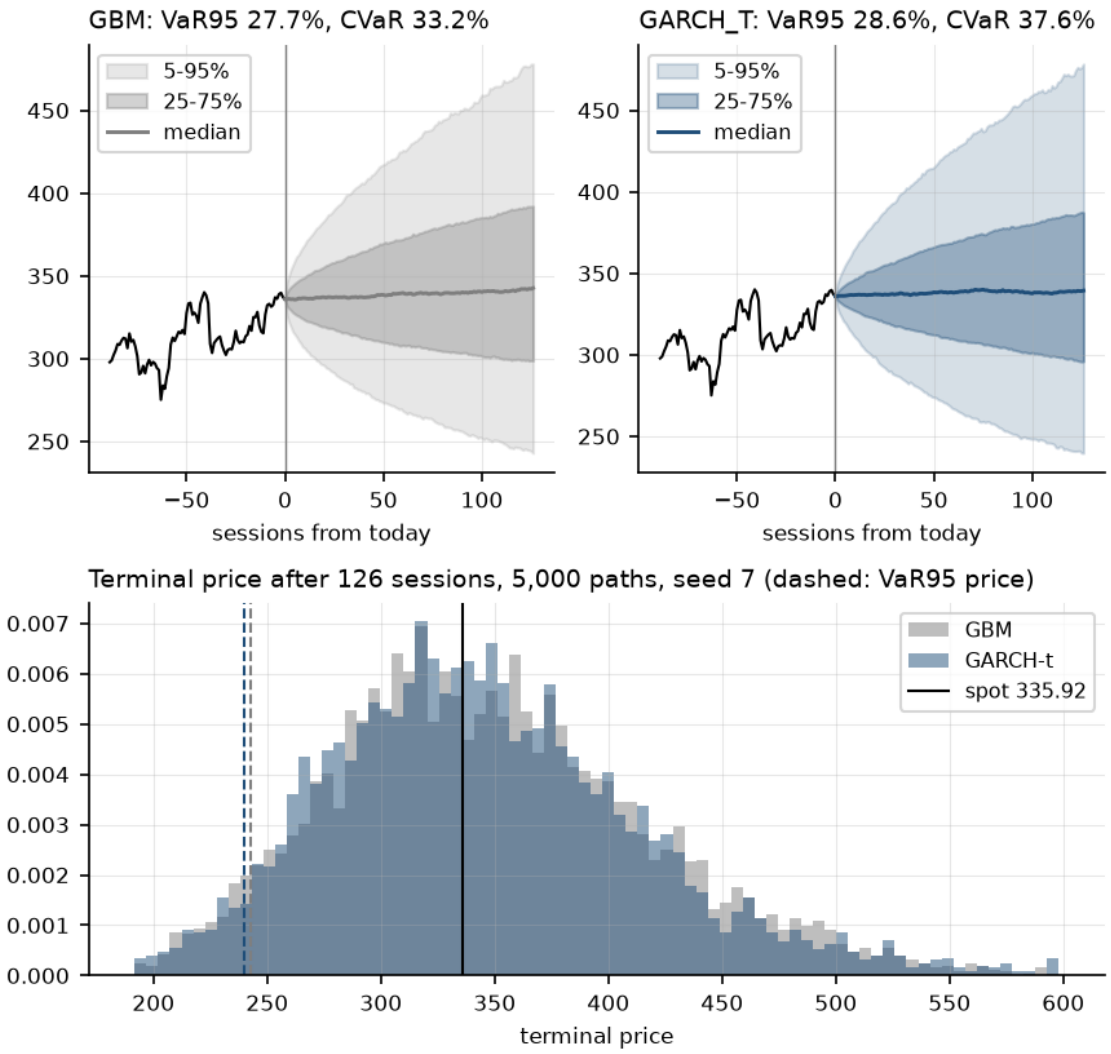



***Figure 5*** *— GBM (left) and GARCH(1,1)-t (right) fans on the same history and the same seed (7), 5,000 paths over 126 sessions: 5-95% and 25-75% bands and the median, after the last 90 sessions of history. Below, the two terminal distributions overlaid, with spot and each model's $VaR_{95}$ price (dashed). GARCH-t starts from today's conditional vol (24.3%) rather than the sample average (28.6%), and its t innovations put more mass in both tails. Source: AAPL daily closes from FMP, 504 sessions, 2024-09-20 to 2026-09-24.*

## 2.5 Probability of loss and of reaching a target

P(loss) is the share of paths whose terminal price is below spot; P(target) is the share at or above the target you typed. They are counts, not model formulas, so they work identically for every model. Worked run: P(loss) = 48% for GARCH-t against 47% for GBM; the median terminal is 339.39 against a spot of 335.92. Note that P(loss) depends on the drift as much as on the volatility: under the zero-drift mode the median sits on spot and P(loss) is close to 50% by construction.

## 2.6 The standard error of a Monte Carlo number

Every Monte Carlo statistic is an estimate from N random paths, and its sampling error falls as 1/√N. For a mean, the standard error is s/√N where s is the standard deviation of the terminal prices; for a percentile the constant is different but the √N is the same. Ten times more paths buys a √10 ≈ 3.2 times smaller error, no more.

Measured on the worked series (GBM, 126 sessions, twenty different seeds): with 500 paths the median terminal moved with a standard deviation of 4.35 and $VaR_{95}$ with 1.32

percentage points; with 5,000 paths 1.10 and 0.46 points. CVaR, which averages only the worst 5% of paths, is noisier: 1.31 points at 500 paths and 0.54 at 5,000. The default of 5,000 paths keeps the tail numbers stable to about a tenth of a point at a cost of a second or two; 500 paths is fine for a quick look at the median but its CVaR is a rough draft. The code clamps paths to [200, 100,000] and the horizon to [5, 1260] sessions.

### 2.7 What changes with the horizon

Under a constant-vol model the cone widens with the square root of the horizon while the drift moves the centre linearly, so the drift matters more and more as the horizon grows: at 21 sessions the fan is almost all volatility; at 252 the drift assumption moves the median by roughly the annual drift. Under GARCH the first sessions are drawn with today's conditional vol, which can be far from the long-run level, and the variance forecast converges to the unconditional level over the half-life (chapter 5.5). VaR at 126 sessions is therefore not the one-day VaR times √126; the app computes each horizon on its own paths.

## 3. Geometric Brownian motion (GBM)

### 3.1 The equation and its discrete step

GBM says the price moves in continuous time with a constant expected growth rate μ and a constant volatility σ, driven by a Brownian motion W:

$$dS = \mu S\,dt + \sigma S\,dW$$

Itô's lemma turns that into a statement about the log price, which is what the app simulates:

$$d\ln S = (\mu - \sigma^2/2)\,dt + \sigma\,dW$$

Over one session (Δt = 1/252) the log return is normal with mean $(\mu - \sigma^2/2)\Delta t$ and standard deviation $\sigma\sqrt{\Delta t}$, so the discretised step is

$$r_t = (\mu - \sigma^2/2)\,\Delta t + \sigma\sqrt{\Delta t}\,z_t, \qquad z_t \sim N(0,\,1)$$

### 3.2 Calibration and the −½σ² term

Calibrating GBM is the sample mean and the sample standard deviation of the daily log returns, annualised as in chapter 1.3. The term $-\sigma^2/2$ exists because the expected value of exp(X) is not exp(E[X]): for a normal X with mean m and variance v, E[exp(X)] = exp(m + v/2). If you want the *price* to grow at rate μ on average, the *log price* must grow at $\mu - \sigma^2/2$; the median price then grows at the lower rate and the mean at μ, the gap being the reward for holding a lognormal asset's right tail.

The app treats the drift as a *log* drift throughout: `_simulate_gbm` draws zero-mean shocks `rng.normal(0.0, sigma * sqrt(dt))` and `run_monte_carlo` adds μ/252 to every step for every model. The $-\sigma^2/2$ therefore appears only inside the drift modes that start from a rate of return on the price (rf and rf + ERP, chapter 8.6), never on top of a sample mean that is already a log mean. On the worked series σ = 28.6% so $\sigma^2/2$ = 4.09% a year: with a 3.00% risk-free rate and a 4.0% premium the premium-mode log drift is +2.91%. The earlier version of the code subtracted the half variance from the sample mean as well, which biased every GBM median down; the test `test_drift_zero_centres_the_cone_and_sample_reproduces_the_mean` now pins the

median to spot × exp(μT) under the sample mode. The $\frac{1}{2}\sigma^2$ is GBM's own correction; the other models subtract their own exact per-step correction under the rate modes (the jump compensator, the bootstrap's block log-MGF, GARCH-t's $c(\sigma_t)$ from each path's conditional variance; chapter 8.6), so the mean price grows at the stated rate under every model.

## 3.3 What GBM gets right and wrong

Right: it is the baseline everyone else is measured against, it has two parameters that any 21-session window can estimate, it never produces a negative price, and it is the model under which Black-Scholes and most of the pricing pages are exact. Wrong, on daily data: volatility is not constant (chapter 4.1), the tails are far fatter than normal (chapter 4.2), large moves cluster in time, and prices sometimes gap. A GBM fan drawn the day after a 6% move is too narrow; one drawn after a placid year is too wide.

## 3.4 Jump diffusion: patching the tails

Merton's model adds a Poisson jump process to the diffusion. The app calibrates it by splitting the return sample at three daily standard deviations: the days beyond 3σ are the jumps, their frequency is the jump intensity λ (jumps per session), their mean and standard deviation set the jump-size distribution, and the remaining days set the diffusion. The compound process's own mean is removed so that the drift mode alone decides the centre. With no outlier in the sample the model degrades to GBM and says so in the jump note. On the worked series the note reads: *Calibrated 6 jump days out of 503 (λ=3.0/yr, mean jump -1.8%).*

## 3.5 Block bootstrap: no distribution at all

The stationary block bootstrap resamples the observed daily returns in blocks of five consecutive sessions (wrapping around the sample), so short-run clustering and the empirical tails survive exactly as far as they exist in the data. Its strength is that it assumes nothing; its weakness is the same thing: it can never draw a move larger than any the window contains, the cone is the sample's own history reshuffled, and the returns are demeaned before resampling so that, again, only the drift mode sets the centre.

## 3.6 The four models side by side

| Model | Assumption | Captures | Misses | When to use |
|---|---|---|---|---|
| GBM | constant μ and σ, normal daily returns | the baseline; a two-parameter fit any window supports | clustering, fat tails, jumps; wrong width right after a shock | as the reference every other model must beat (dAIC vs GBM); pricing logic |
| Jump diffusion | GBM plus Poisson jumps calibrated from the >3σ days | gap risk; a fat left tail when the sample has one | clustering; needs outliers in the window to be different from GBM | a name with known event risk (earnings, policy) when the window contains such days |
| Block bootstrap | the future is a reshuffle of the past 5-session blocks | the empirical tails and short-run clustering, with no formula | anything the window never showed; long-memory vol; a real forecast of today's vol | a sanity check on the parametric models; series whose shape you distrust |
| GARCH(1,1)-t | variance follows its own recursion; innovations are Student-t | clustering, fat tails, a fan that starts from today's vol and mean-reverts | jumps as such; asymmetry (down moves raising vol more than up moves) | the default: daily equity, FX and rates series with at least a few hundred sessions |

# 4. Volatility clustering and fat tails

Two facts about daily financial returns hold across almost every market and decade, and GBM has neither. They are the reason the Quant Lab defaults to GARCH-t.

## 4.1 Clustering: large moves follow large moves

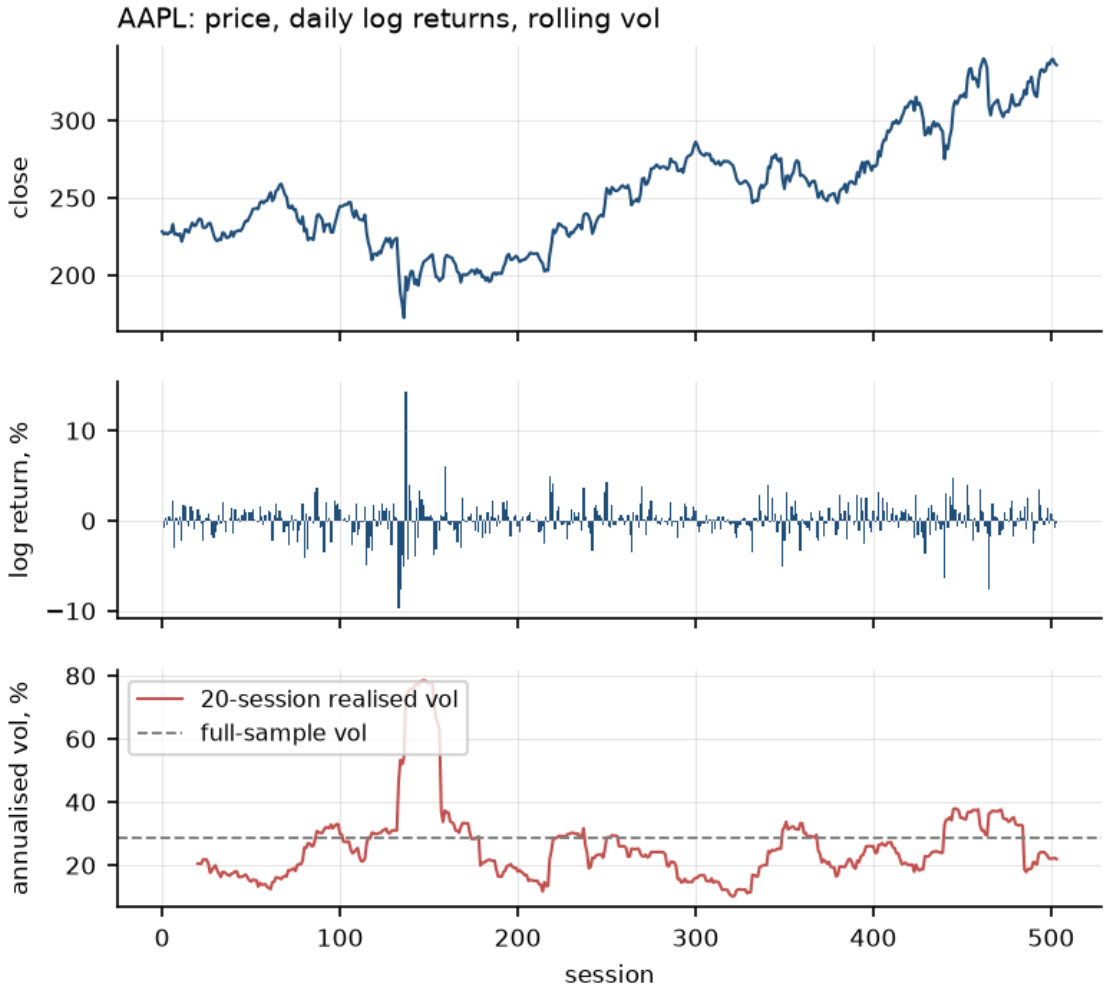



***Figure 1*** *— AAPL — price, daily log returns and the 20-session realised volatility (annualised) against the full-sample volatility of 28.6%. Calm and turbulent stretches are visible in the return panel and as swings of the rolling vol. Source: AAPL daily closes from FMP, 504 sessions, 2024-09-20 to 2026-09-24.*

Returns themselves are almost unpredictable: the lag-1 autocorrelation of $r_t$ on the worked series is +0.040. Their *squares* are not: the autocorrelation of $r_t^2$ is +0.222 at lag 1, +0.089 at lag 5 and +0.047 at lag 20. Knowing yesterday's move was big does not tell you the sign of today's, but it does tell you today's is likely to be big too. That is volatility clustering, first modelled by Engle (1982), and it is precisely the structure the GARCH recursion in chapter 5 is built to follow.

## 4.2 Fat tails: excess kurtosis

Kurtosis measures how much of a distribution's variance comes from its extremes. For standardised returns z (mean zero, variance one) the **excess kurtosis** is

$$\kappa = \mathrm{E}[z^4] - 3$$

which is zero for the normal distribution. The risk report in `core/quant/risk.py` prints it as `mean(z**4) - 3`, and its narrative says "Excess kurtosis of X means fat tails" whenever X exceeds 2. On the worked series the raw returns have $\kappa = 10.33$ (skewness +0.26). A normal distribution with the same variance would put far fewer days beyond three standard deviations than the data contain.

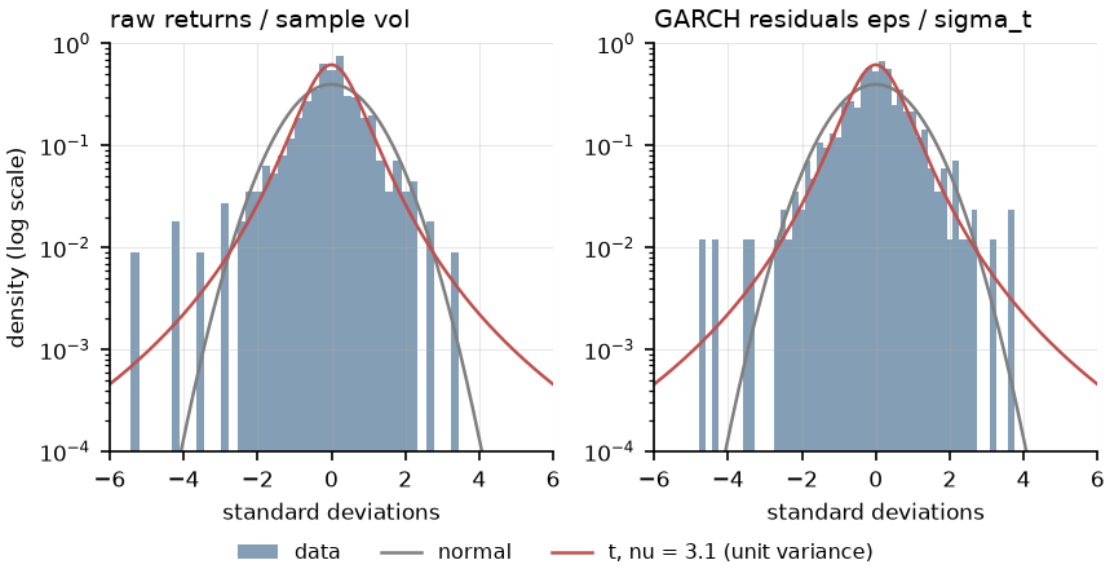



***Figure 2*** *— Histogram of standardised returns (log scale) against the standard normal density and a unit-variance Student-t with $\nu = 3.1$ from the fitted model. Left: raw returns divided by the sample volatility. Right: the same returns divided by the GARCH conditional volatility $\sigma_t$ — the tails that remain are what $\nu$ has to explain. Source: AAPL daily closes from FMP, 504 sessions, 2024-09-20 to 2026-09-24.*

Fat tails in raw returns have two sources. The first is time-varying volatility: a mixture of calm days and wild days looks fat-tailed even if each day is normal given its own $\sigma_t$. The second is genuinely fat-tailed innovations. GARCH separates them: after dividing each return by the fitted $\sigma_t$, the excess kurtosis of the residuals falls from 10.33 to 5.31. What is left is handled by the Student-t with $\nu = 3.1$ (chapter 6), whose own excess kurtosis $6/(\nu - 4)$ is infinite ($\nu \le 4$).

### 4.3 Why a constant-vol fan is wrong in both directions

A GBM fan uses one volatility for every future session: the sample average, 28.6% here. GARCH-t starts from today's conditional vol, 24.3%, and lets it decay towards the unconditional 33.1% over a half-life of 3.7 sessions, for an average of 31.3% over the 126-session horizon. Right after a shock the conditional vol sits above the average and a constant-vol fan is too narrow: it understates the VaR of the next few weeks. After a long calm the conditional vol sits below the average and the constant-vol fan is too wide, which sounds harmless until it makes a hedge look unnecessary or a target look unreachable. The difference shrinks with the horizon, because the GARCH forecast converges to the same long-run level; it never disappears when the persistence is high.

## 5. GARCH(1,1)

### 5.1 The equation, term by term

Bollerslev's (1986) GARCH(1,1) keeps the return equation of chapter 1 and gives the variance its own recursion:

$$r_t = \mu + \varepsilon_t, \qquad \varepsilon_t = \sigma_t z_t, \qquad \sigma_t^2 = \omega + \alpha\,\varepsilon_{t-1}^2 + \beta\,\sigma_{t-1}^2$$

Today's variance is a constant plus a share of yesterday's squared surprise plus a share of yesterday's variance. In words: **ω** (omega) is the floor, the part of the variance that is always there; **α** (alpha) is the reaction to yesterday's surprise, how much a large move raises tomorrow's variance; **β** (beta) is the memory, how much of yesterday's variance carries over regardless of what happened. The mean μ is the sample mean of the returns and is held fixed during the fit (`mu = float(np.mean(r))`): a 500-session sample cannot pin down the mean anyway, and letting the optimiser chase it only makes the volatility parameters noisier.

The worked fit: ω = 7.467e-05 (daily variance units), α = 0.183, β = 0.645, on T = 503 returns with μ fixed at +0.077% per day. The app shows α and β to three decimals on the second stat strip, each followed by its 95% interval in brackets (chapter 5.8).

## 5.2 Persistence and the unconditional variance

Taking expectations of the recursion and noting $E[\varepsilon^2] = E[\sigma^2]$ in the long run gives the **unconditional variance**, the level the variance settles at when nothing new happens:

$$\bar{\sigma}^2 = \frac{\omega}{1 - \alpha - \beta}$$

The sum α + β is the **persistence**. The worked fit has α + β = 0.8287, so $\bar{\sigma}^2$ = 7.467e-05 / (1 − 0.8287) = 4.360e-04 per day, i.e. an unconditional vol of √(4.360e-04 × 252) = 33.1% a year; the "Uncond vol (ann.)" tile. As α + β approaches 1 the denominator goes to zero and the unconditional variance goes to infinity: the model no longer has a long-run level to return to. The code guards this in two ways (chapter 7.6): the objective refuses α + β > 0.9995 outright, and any fit with α + β > 0.995 is flagged "stationarity" and coloured red.

## 5.3 The half-life of a variance shock

A deviation of the variance from its long-run level decays by the factor (α + β) each session, so the number of sessions for it to halve is

$$\text{half-life} = \frac{\ln 0.5}{\ln(\alpha + \beta)}$$

For the worked fit that is ln 0.5 / ln 0.8287 = 3.7 sessions, the "Half-life (sessions)" tile. A half-life of 5 sessions means the market forgets a shock in a week; 50 sessions means a quarter; the code reports infinity when the persistence is 1 or more.

## 5.4 The conditional volatility path

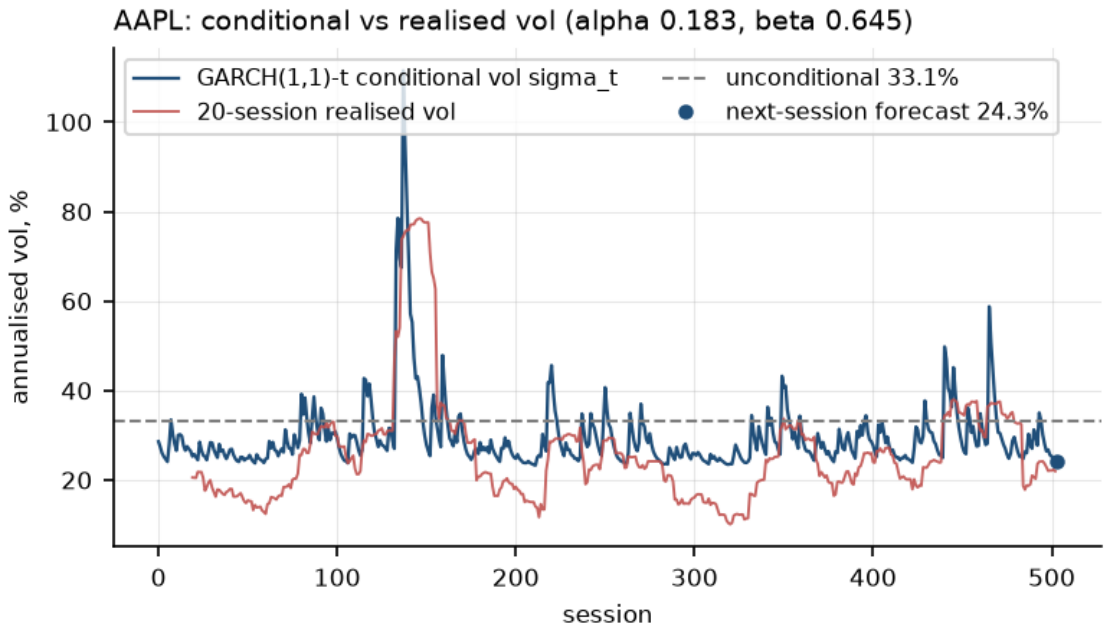



***Figure 3*** *— AAPL — the fitted conditional volatility $\sigma_t$ (annualised) against the 20-session realised volatility and the unconditional level 33.1%. The dot is $\sigma_{T+1}$, the next-session forecast (24.3%) that the fan starts from. Source: AAPL daily closes from FMP, 504 sessions, 2024-09-20 to 2026-09-24.*

The recursion needs a starting value. The code starts $\sigma_1^2$ at the sample variance of ε, i.e. at the unconditional level, and then runs forward through the whole sample (`_variance_path`, implemented as a first-order IIR filter with `scipy.signal.lfilter`: $y_t = x_t + \beta\, y_{t-1}$ with $x_t = \omega + \alpha\, \varepsilon_{t-1}^2$). The last element of that path is $\sigma_{T+1}^2$, the one-session-ahead forecast; annualised it is the "Cond vol (ann.)" tile and the volatility the first simulated session is drawn with. The realised vol in the figure lags the conditional vol because it averages the past 20 days with equal weights, while GARCH weights recent days geometrically.

## 5.5 Forecasting the variance h sessions ahead

Beyond the next session the expected squared surprise equals the expected variance, so the forecast recursion collapses to a single line:

$$\sigma^2_{T+h} = \omega + (\alpha+\beta)\,\sigma^2_{T+h-1}, \qquad \text{equivalently}$$
$$\sigma^2_{T+h} = \bar{\sigma}^2 + (\alpha+\beta)^{h-1}\,(\sigma^2_{T+1} - \bar{\sigma}^2)$$

This is `forecast_variance`: it starts from $\sigma_{T+1}^2$ and applies $\omega + (\alpha + \beta)\,\sigma^2$ $h - 1$ times. The **horizon vol** the app reports is the square root of the average forecast variance over the horizon, annualised (`horizon_vol`): 31.3% for 126 sessions on the worked fit, between today's 24.3% and the long-run 33.1%. The acceptance test `test_garch_forecast_variance_converges_to_unconditional` checks that the first step equals the fit's forecast, that step 5,000 is within 1% of $\omega/(1-\alpha-\beta)$, and that the horizon vol lies between the two.

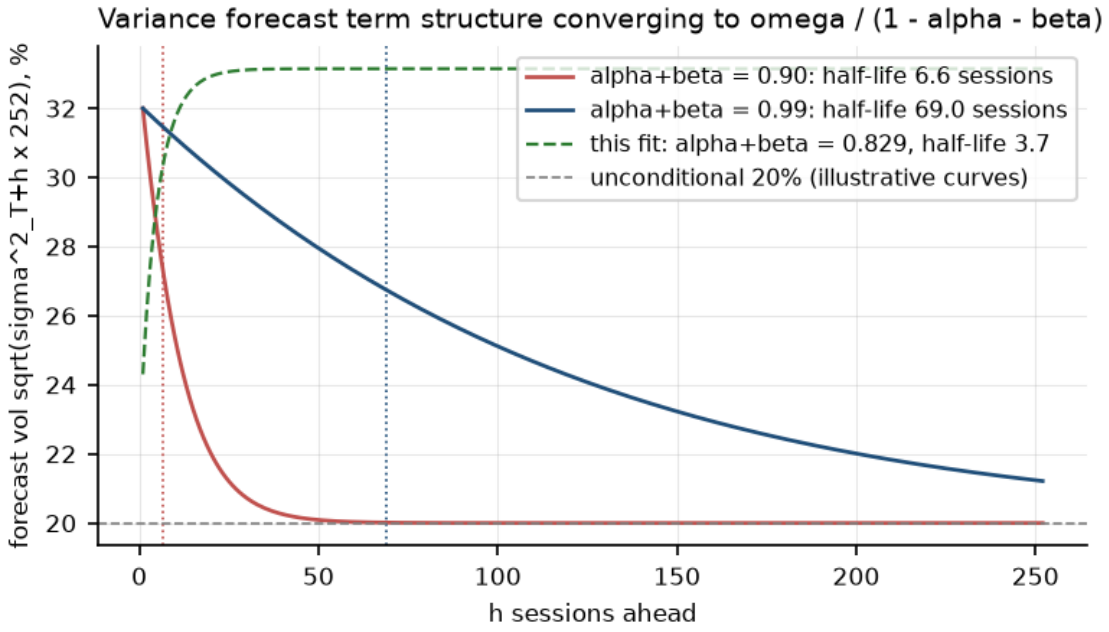



***Figure 4*** *— Forecast volatility $\sqrt{(\sigma_{T+h}^2 \times 252)}$ for h = 1 ... 252, starting at 32% and converging to an unconditional 20%, for persistence 0.90 (half-life 6.6 sessions) and 0.99 (half-life 69 sessions); dotted verticals mark the half-lives. Dashed green: the worked fit's own term structure (α + β = 0.829, half-life 3.7).*

## 5.6 IGARCH and the near-integrated flag

When α + β = 1 the model is called integrated GARCH (IGARCH): the unconditional variance is undefined, the half-life is infinite, and the variance forecast stays at today's level forever (with ω > 0 it drifts up without bound). Daily data often push the fit close to that line because a short window with one regime change looks like permanent variance. The code enforces α + β ≤ 0.9995 by returning a penalty of $-10^{10}$ from the objective, and reports "stationarity" in `at_bound` for any fit above 0.995; the narrative then says "near-integrated: the sample cannot pin down where vol settles; use a longer history". The fan of such a fit is honest for a few weeks and meaningless at a year, because its width never mean-reverts.

## 5.7 Typical values and what unusual ones mean

| Parameter | Typical on daily equity data | If it is unusual |
|---|---|---|
| α | 0.05 – 0.10 | near 0: no reaction to shocks, GARCH adds nothing (see the GBM demo series in chapter 7); at 0.5 (the cap): a few outliers dominate — look at them, try the jump model |
| β | 0.85 – 0.93 | near 0: no memory, the series is closer to iid; at 0.999: integrated, see 5.6 |
| α + β | 0.95 – 0.99 | > 0.995: flagged near-integrated; the unconditional vol tile is unreliable and the horizon vol barely decays |
| half-life | 10 – 70 sessions | under 5: the model forgets within a week and the fan is essentially GBM after that; over 200: the fan never returns to the long-run level |
| ω | $10^{-6}$ – $10^{-5}$ (daily variance) | only meaningful together with α + β; the tile shows the unconditional vol instead |

## 5.8 How precisely are α, β and the persistence known?

A fitted parameter is an estimate from one sample; another two years would have given another number. Maximum likelihood carries its own measure of that uncertainty: at the optimum, the curvature of the log-likelihood says how sharply the data pin each parameter down. Minus the matrix of second derivatives of ln L at the fit is the **observed information**; its inverse is the covariance of the estimates, the square roots of its diagonal are the **standard errors**, and a 95% interval is the estimate plus or minus 1.96 standard errors (the Wald interval):

$$\mathrm{Cov}(\hat{\theta}) \approx [\, -H(\hat{\theta})\,]^{-1}, \qquad H = \text{second derivatives of } \ln L, \qquad \hat{\theta} \pm 1.96\,\mathrm{SE}(\hat{\theta})$$

The code takes the second derivatives numerically (central differences in the optimiser's scaled coordinates, one-sided where a parameter sits on a bound), checks that the matrix is positive definite, and inverts it (`_hessian`, `_covariance` in `core/quant/garch.py`). The persistence α + β gets its standard error from the same covariance by the delta method (its gradient is (0, 1, 1, 0)). The half-life interval is the persistence interval mapped through ln 0.5 / ln p: the map is monotone, so the half-life interval covers the truth exactly as often as the persistence interval does, and it can never go negative, which a symmetric interval around a short half-life would.

Worked fit: α = 0.183 (-0.013 to 0.380), β = 0.645 (0.283 to 1.007), α + β = 0.8287 (0.5656 to 1.0918), half-life 4 (1 to inf) sessions; standard errors 0.100, 0.185 and 0.134. The tiles show exactly these strings: "0.183 (-0.013 to 0.380)", "0.645 (0.283 to 1.007)", "0.8287 (0.5656 to 1.0918)", "4 (1 to inf)".

On this window the interval for alpha and beta crosses a bound (0 and 1): 503 sessions determine the tail of this series far better than its dynamics, and the red chip "interval touches a bound: alpha, beta" says so. A symmetric interval on a parameter that lives near a bound is indicative only; the estimate itself is still the maximum-likelihood one. The persistence interval reaches 1, which is the case for most two-year samples: they do not exclude an integrated variance at 95%, and the half-life's upper end is therefore infinite. The app shows this as a muted chip, "persistence interval reaches 1", not as an error, because it is the normal state of a 500-session fit rather than a failure of this one.

Three things the intervals do not do, each measured before the feature was built (`docs/B2_FIX_BULLETIN.md`):

- **The unconditional vol carries no interval on purpose.** Its delta-method interval was tried on simulated series and covered the truth 42% of the time when the fitted persistence was below 0.95, and 100% of the time, with an absurd width (a median standard error of 297 vol points), above 0.995. No threshold rescues it, so the near-integrated chip carries that message instead and the tile's tooltip says why.
- **They describe sampling error, not model error.** On series whose innovations were a skewed mixture rather than a t, the interval for the tail parameter covered the truth 76% of the time: the standard error was the right size and the estimate was biased. The intervals assume the t is the right family.
- **They can be undefined, and the failure is informative.** When a difference step leaves the domain (a near-integrated fit next to the stationarity cap) or the information matrix is not positive definite, the chip "SE undefined" appears with the reason and the tiles show the values alone. The demo series does exactly that: with α at 0 its β is unidentified and the note reads *the information matrix is not positive definite at the*

*optimum*. The profile interval of chapter 6.5 still exists in that case.

How good are they? On 50 simulated GARCH(1,1)-t series of 1,000 sessions the 95% intervals covered the true α, β and persistence 96%, 96% and 100% of the time; the acceptance test `test_interval_coverage_on_simulated_t_garch` asserts at least 85%. On 500 sessions the same study measured 91% to 97%, with α the short one: a symmetric interval on a positive parameter near 0 is the weak spot, and the bound chip is its warning.

# 6. Student-t innovations

## 6.1 The t distribution and ν

The Student-t distribution with ν (nu) degrees of freedom is bell-shaped like the normal but with heavier tails; as ν grows it converges to the normal (at ν = 30 the two are practically indistinguishable, and above ν = 100, where the excess kurtosis is under 0.07, the app draws and displays the innovations as normal). Its variance is ν/(ν − 2), finite only for ν > 2, and its excess kurtosis is

$$\kappa_t = \frac{6}{\nu - 4}, \qquad \text{finite only for } \nu > 4$$

so a t with ν = 5 has excess kurtosis 6, ν = 8 gives 1.5, ν = 4 gives infinity. In the model the innovation $z_t$ is a t draw scaled to unit variance, so ν describes only the shape of the surprises, not their size — the size is $\sigma_t$'s job. Bollerslev (1987) introduced this pairing; it is the "-t" in GARCH(1,1)-t.

## 6.2 Scaling to unit variance: the classic bug

Because a raw t draw has variance ν/(ν − 2) rather than 1, feeding it straight into $\varepsilon_t = \sigma_t z_t$ would inflate every variance by that factor and make ω, α, β and every VaR wrong by an amount that depends on ν. The code scales:

$$z_t = t_\nu \times \sqrt{\frac{\nu - 2}{\nu}}$$

```
def _draw_z(rng, nu, dist, size, bound=Z_BOUND):        # Z_BOUND = 22
    if dist != "t" or nu > NU_NORMAL:                       # normal, or a t drawn as one
        return rng.standard_normal(size=size)
    s = math.sqrt((nu - 2.0) / nu)
    z = rng.standard_t(nu, size=size) * s
    ...                 # |z| > bound: redrawn from the t restricted to [-bound, bound]
    return z
```

Simulated t innovations are also capped at ±22: a draw beyond the cap is replaced by one from the same t restricted to the cap, nothing is rescaled, so inside the cap the law is the fitted density. The cap exists because under an unbounded t the simulated price has no finite mean and no risk-neutral drift can exist; the Quant Lab's bound note (chapter 9.5) states what the cap removes, and the horizon-vol tile describes the paths actually drawn.

The same factor appears inside the likelihood (chapter 7.3): the density is written for the scaled variable, so the fitted $\sigma_t$ is a true standard deviation. Getting one of the two places right and the other wrong is the classic mistake, and it is silent: the fit still converges, the numbers still look plausible. Two checks exist for it. The self-check in `python -m core.quant.garch` draws 200,000 scaled innovations and asserts their variance is within 3% of 1; the acceptance test `test_garch_scaled_t_shocks_have_unit_variance` simulates with α = β = 0 (so $\sigma^2 = \omega$ exactly) and asserts the sample variance over ω is 1 and the kurtosis is still fat.

## 6.3 Reading ν, and the η = 1/ν parameterisation

The fit does not search over ν directly but over η = 1/ν on [0, 0.4]. Two things follow. η = 0 *is* the normal model: as ν → ∞ the t density converges to the normal term by term, so the normal GARCH is a point inside the t model's parameter space rather than a limit it can never reach. And η = 0.4 is ν = 2.5, the heaviest tail allowed. There is no upper bound on ν any more. A fit that lands at η = 0, or at a ν above 100, is reported as "normal innovations" on a muted chip, the tile reads "normal", and the sampler draws normal innovations: that is an answer, not a failure. Before this parameterisation ν was capped at 30, the cap excluded the normal from the t model, and a series with no heavier tail sat on the cap with a red chip and a likelihood-ratio statistic that could come out negative.

On daily equity data ν typically lands between 3 and 8: clearly fatter than normal. The worked fit gives ν = 3.1 (excess kurtosis of the innovations infinite, ν ≤ 4). A ν at 2.5 (η at 0.4) means the sample wants tails heavier than the model allows, usually a handful of extreme days dominating a short window; the variance of the innovation is still finite there but only just, and the fit is fragile. The screen shows "at bound: nu" and the narrative says "tails heavier than the model allows". The app's demo series, a GBM path, lands on the normal: its fit reports ν = 515, effectively normal with the note *GARCH(1,1)-t on 503 sessions, mu fixed at -0.084%/day: alpha=0.000 beta=0.991 persistence=0.991 nu=514.5 (10.9 to normal); normal innovations (nu = 515 > 100); alpha at bound; standard errors undefined: the information matrix is not positive definite at the optimum.*

## 6.4 The likelihood-ratio test: t versus normal, with the null on the boundary

The t model has one more parameter than the normal GARCH and contains it at η = 0. Twice the gain in log-likelihood from the extra parameter is the likelihood-ratio statistic:

$$\mathrm{LR} = 2\left(\ln L_t - \ln L_{\mathrm{normal}}\right) \geq 0$$

It cannot be negative, because the best t fit is at least the best normal fit. The code makes sure of it: the t fit is seeded from the normal optimum (with η = 0), the normal is then re-fitted from the t's (ω, α, β) so the reference is the best normal reachable through the same code path, and the statistic is clamped at 0 against optimiser tolerance. The null hypothesis η = 0 lies on the *boundary* of [0, 0.4], so the textbook $\chi^2_1$ result does not apply: about half the time the fit lands exactly on the normal and LR = 0, and the null distribution is the mixture (Self and Liang, 1987)

$$\mathrm{LR} \sim \tfrac{1}{2}\,\delta_0 + \tfrac{1}{2}\,\chi^2_1, \qquad 5\%\ \text{critical value } 2.706$$
$$(3.841,\ \text{the } \chi^2_1 \text{ value, is its } 2.5\%\ \text{point})$$

`fit_garch` fits both models on the same data and stores `lr_t_vs_normal`. The screen shows it as the "LR t vs normal" tile, green when it exceeds `LR_5PCT = 2.706` and grey otherwise; it is greyed as well when α sits at its bound, because the mixture assumes α and β interior. The narrative says "innovation tails heavier than normal" or "the t tail did not beat a normal GARCH on this sample". Worked fit: $\ln L_t$ = 1391.8, $\ln L_{\mathrm{normal}}$ = 1349.3, LR = 85.0, significant at 5%. On the GBM demo series LR = 0.0: nothing to gain from the t, as it should be. The acceptance test `test_lr_is_nonnegative_and_correctly_sized_under_the_null` fits 200 normal-innovation series and checks that LR is never negative, that between 30% and 70% of the fits land exactly at η = 0, and that the share above 2.706 stays within sampling error

of 5%.

## 6.5 How precisely is ν known? The profile interval

A symmetric interval is the wrong shape for ν. The likelihood is flat towards the normal (a ν of 20 and a ν of 200 are nearly the same model) and steep towards 2.5, and the fitted coordinate η has a bound at 0 where the normal lives. The app therefore inverts the likelihood-ratio test instead of using a standard error: it re-maximises ln L over (ω, α, β) at each η on a grid from 0 to 0.4, which gives the **profile likelihood**, and keeps every η at which the profile stays within a cut of the maximum:

$$\left\{ \eta : \max_{\omega,\,\alpha,\,\beta} \ln L(\omega, \alpha, \beta, \eta) \geq \ln L_{\max} - c \right\}$$

$$c = 1.92 \text{ inside } (0,\ 0.4], \qquad c = 1.353 \text{ at } \eta = 0$$

The inside cut 1.92 is half the $\chi^2_1$ 95% point (3.841/2). At η = 0 the null is the boundary mixture of chapter 6.4, so the cut there is half of 2.706, and the interval reaches "normal" exactly when the LR tile says the t did not beat the normal; between 0 and the first grid point the cut ramps between the two values so the end moves continuously with the data. The ends are mapped back by ν = 1/η, so the range is asymmetric and can be one-sided. Worked fit: ν = 3.1 (2.5 to 4.4); the tile reads "3.1 (2.5 to 4.4)". Demo series: >100 (10.9 to normal). Figure 6 shows both profiles with their cuts and intervals.

Reading. The lower end sits on the ν floor 2.5: the sample would like tails at least as heavy as the model allows, and the chip "interval touches a bound: nu" says so; the upper end 4.4 is the honest one. On 50 simulated series of 1,000 sessions with a true ν of 6 the profile interval covered the truth 90% of the time and its median ran from 4.3 to 9.0: four years of daily data identify ν to within a factor of two, no better, and two years to within a factor of three. A ν shown to one decimal is a point estimate; the bracket is what the sample knows.

The scan has a second use. Each profile point is a full refit of (ω, α, β) at a fixed η, so if any of them beats the joint fit's log-likelihood the optimiser had stalled short of the maximum; `fit_garch` then re-seeds the joint fit from that point (chapter 7.5). The check found a dead starting value that had been in the code since the beginning. Each refit warm-starts from the previous grid point, and where the result lands outside the cut, which is where an end of the interval is decided, it is retried from the joint optimum: a single warm-started run can stall on a flat ridge (the demo series, with α = 0, has one), which would dent the profile and narrow the interval. Figure 6 runs both starts at every point.

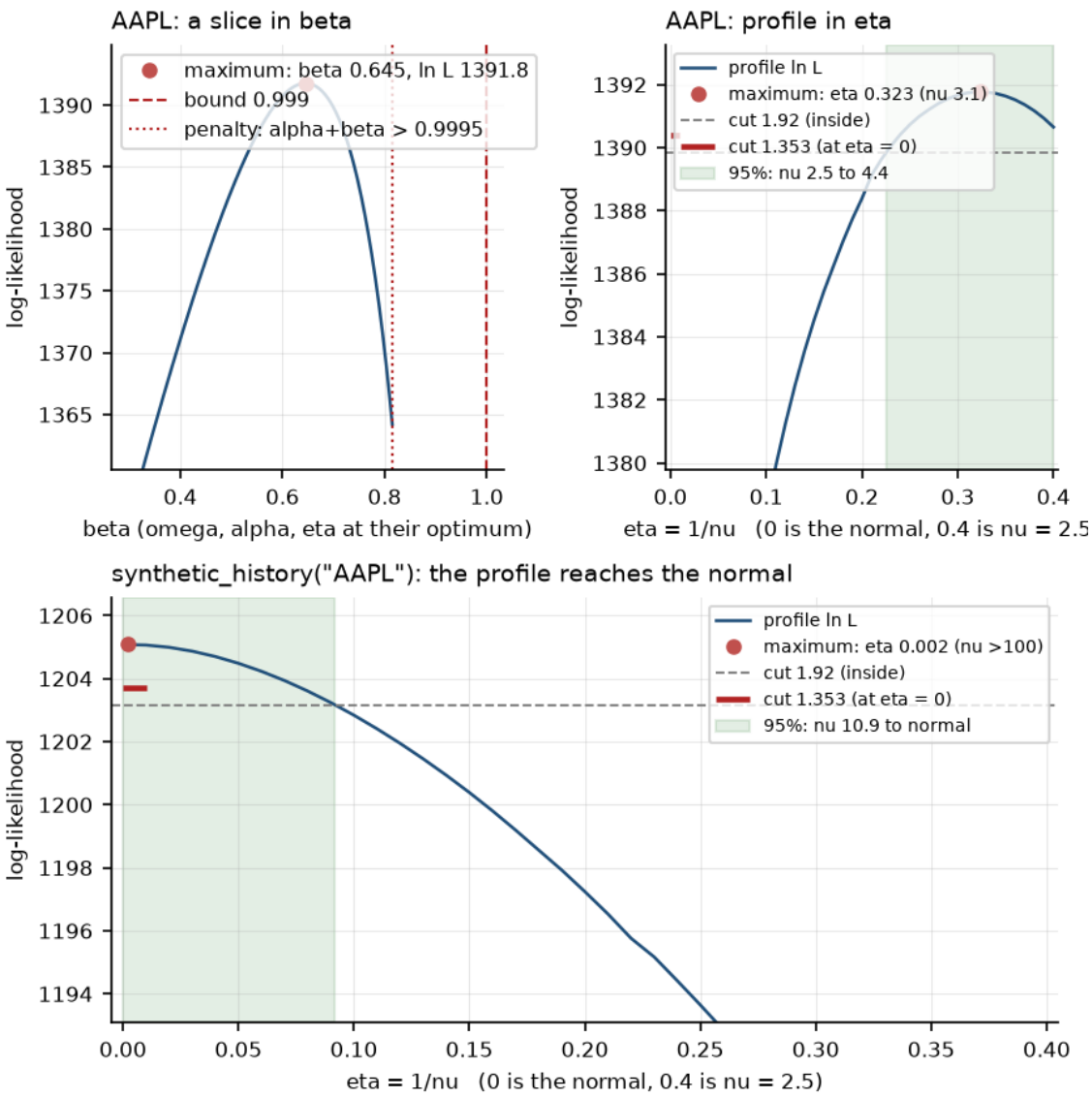



***Figure 6*** *— Top left: the log-likelihood of the worked series as a function of β with ω, α and η held at their optimum; the dot is the fitted maximum (β = 0.645), the dashed line the bound at 0.999, the dotted line where the stationarity penalty starts. Top right: the profile log-likelihood of the worked series in η = 1/ν, with the inside cut (dashed), the boundary cut at η = 0 (red tick) and the 95% interval (shaded): ν 2.5 to 4.4. Bottom: the same for the app's GBM demo series, synthetic_history("AAPL"): the maximum sits at the normal end and the interval runs from ν = 10.9 to normal, so the tile says "normal".*

# 7. Maximum likelihood, log-likelihood, and the guardrails

## 7.1 What a likelihood is

Fix a set of parameters θ = (ω, α, β, η), η = 1/ν. The model then says how probable each observed return was: the density of $r_t$ given the variance the recursion assigns to that day. Multiply those densities over all T days and you have the probability density of the whole sample, *as a function of θ*. That function is the likelihood L(θ). Maximum likelihood picks the θ under which the data you actually saw were most probable. It is not the probability that θ is true; it is a score, and the fit is the θ with the best score.

## 7.2 Why the log

A product of 500 densities around 40 each is $40^{500}$, far outside what a floating-point number can hold, and its derivative is unmanageable. Taking logs turns the product into a sum, keeps every term in a sane range, and leaves the maximiser unchanged because the log is increasing. Every number the app calls "loglik" is ln L, and differences of ln L (not ratios of L) are what the LR test and AIC use.

## 7.3 The per-observation formulas, as implemented

With $\varepsilon_t = r_t - \mu$ and the variance path $\sigma_t^2$ from the recursion, the normal GARCH contributes, per day,

$$\ln L_t = -\tfrac{1}{2}\ln(2\pi) - \tfrac{1}{2}\ln\sigma_t^2 - \frac{\varepsilon_t^2}{2\,\sigma_t^2}$$

and the unit-variance Student-t contributes

$$\ln L_t = c(\nu) - \tfrac{1}{2}\ln\sigma_t^2 - \tfrac{\nu+1}{2}\ln\!\left(1 + \frac{\varepsilon_t^2}{(\nu-2)\,\sigma_t^2}\right),$$

$$c(\nu) = \ln\Gamma\!\left(\tfrac{\nu+1}{2}\right) - \ln\Gamma\!\left(\tfrac{\nu}{2}\right) - \tfrac{1}{2}\ln\left(\pi\,(\nu-2)\right)$$

The $(\nu - 2)$ in the denominator is the unit-variance scaling of chapter 6.2 written into the density. As $\eta = 1/\nu \to 0$ the t term converges to the normal term and $c(\nu)$ to $-\tfrac{1}{2}\ln(2\pi)$, so at $\eta = 0$ the code uses the normal formula outright: the t contains the normal exactly, which is what makes the likelihood ratio of chapter 6.4 non-negative. The code is a direct transcription; `_t_const` is $c(\nu)$ (`gammaln` is $\ln\Gamma$) and `np.log1p(x)` is $\ln(1 + x)$ computed accurately for small x:

```
if dist == "t" and params[3] >= _ETA_EPS:            # eta = 1/nu; eta = 0 IS the normal
    nu = 1.0 / params[3]
    ll = (_t_const(nu) - 0.5 * np.log(s2)
          - ((nu + 1) / 2) * np.log1p(eps2 / ((nu - 2) * s2)))
else:                                                # normal, or a t with eta = 0
    ll = -0.5 * math.log(2 * math.pi) - 0.5 * np.log(s2) - eps2 / (2 * s2)
return float(np.sum(ll))
```

Above $\nu = 10^4$ the two $\ln\Gamma$ terms of $c(\nu)$ are each of order $\nu$ and cancel catastrophically (at $\nu = 10^8$, the finite-difference step the optimiser takes from $\eta = 0$, they produced spurious gradients of $2 \times 10^{-5}$), so `_t_const` switches there to the Stirling expansion $\tfrac{1}{2}\ln(\nu / (2\pi(\nu - 2))) - 1/(4\nu)$. The GBM benchmark is the same normal formula with one constant variance, the sample variance with ddof = 1, and the sample mean (`loglik_iid_normal`). Three things are checked before the sum: $\alpha + \beta$ above the stationarity cap returns the penalty $-10^{10}$; any non-finite or non-positive variance on the path returns the same penalty; and $\eta \le 0.4$ keeps $\nu \ge 2.5$, so $\nu - 2$ never reaches zero.

## 7.4 AIC and BIC

A model with more parameters always reaches a higher likelihood on the sample it was fitted to. Information criteria charge for the extra parameters:

$$\mathrm{AIC} = 2k - 2\ln L, \qquad \mathrm{BIC} = k\ln T - 2\ln L$$

where k is the number of estimated parameters (k = 4 for GARCH-t: ω, α, β, η; 3 for the normal GARCH; 2 for GBM: mean and variance) and T the number of returns. Lower is

better; BIC penalises parameters harder as T grows. The app shows **ΔAIC vs GBM** = $AIC_{GBM} - AIC_{GARCH\text{-}t}$: positive means GARCH earned its parameters. Worked fit: $AIC_{GARCH\text{-}t} = 2\times4 - 2\times1391.8 = -2775.5$, $AIC_{GBM} = 2\times2 - 2\times1306.9 = -2609.7$, ΔAIC = +165.8; BIC = -2758.7. A rule of thumb: a ΔAIC above 10 is decisive, between 2 and 10 is meaningful, below 2 is a tie. On the GBM demo series ΔAIC = -3.9: the tile is grey and the narrative says "GBM is not beaten here", which is the correct answer for a series that *is* GBM.

## 7.5 Numerical optimisation: L-BFGS-B, starts, bounds, convergence

There is no formula for the θ that maximises ln L; a numerical optimiser climbs the surface. The code uses SciPy's L-BFGS-B, a quasi-Newton method that accepts box bounds on each parameter, and minimises −ln L. Five details matter:

- **Starting values.** $\alpha_0 = 0.05$, $\eta_0 = 1/8$ ($\nu_0 = 8$), and $\omega_0 = \text{var}(\varepsilon)(1 - \alpha_0 - \beta_0)$ so that the start already has the sample variance as its unconditional level. $\beta_0$ is tried at 0.80, 0.90 and 0.94 — three runs, keep the best (the multi-start). The surface has a long ridge along $\alpha + \beta \approx$ constant and flat regions in η; one start can stop on a shoulder. The third value used to be 0.95, which with $\alpha_0 = 0.05$ started exactly on the stationarity penalty with $\omega_0 = 0$, so it was dead and some 500-session fits stalled below the maximum; the profile scan of chapter 6.5 caught it and now re-seeds the fit whenever one of its refits beats the joint fit. The t fit is also seeded from the normal optimum (chapter 6.4).
- **Scaling.** ω is optimised in units of $10^{-6}$ (`_OMEGA_SCALE`). At its natural scale, $\sim10^{-6}$, the finite-difference gradient in ω swamps the others and L-BFGS-B stops after a handful of iterations with α still at its start. The comment in `_fit` records this; it was a real bug.
- **Bounds.** `BOUNDS` = $\omega \in [1e\text{-}12, \infty)$, $\alpha \in [0.0, 0.5]$, $\beta \in [0.0, 0.999]$, $\eta \in [0, 0.4]$ ($\nu \geq 2.5$, no upper bound), plus the stationarity penalty at $\alpha + \beta > 0.9995$. A parameter within $10^{-4}$ (relative; absolute at zero) of a bound is listed in `at_bound`; η at 0.4 is listed as "nu", and η at 0 is not a bound hit but the normal (chapter 6.3).
- **Convergence.** `converged` is the optimiser's own success flag for the best start. False means it stopped for a reason other than "no further improvement": iteration limit, a line search that failed, a non-finite objective. The screen shows a red chip.
- **Precision.** After the optimum the code takes the observed information for the Wald intervals (chapter 5.8) and scans the profile in η for the ν interval (chapter 6.5); together they cost about as much as the fit itself. `fit_garch(profile=False)` skips the scan for callers that fit in loops.

## 7.6 The guardrails, one by one

Each row is one guardrail in `core/quant/garch.py`: what it is, why it exists, what the screen shows when it binds, what to do, and whether it is safe to change. The screen strings are quoted from `ui/quant_lab_page.py` and `core/quant/insights.py`.

| Guardrail | What it is | Why it exists | What you see when it binds | What to do | Safe to change? |
|---|---|---|---|---|---|
| ω ≥ 1e-12 (BOUNDS["omega"]), optimised in units of $10^{-6}$ | A strictly positive floor on the variance constant. | ω ≤ 0 makes the variance path zero or negative and ln $\sigma^2$ undefined; the scaling keeps the ω gradient comparable to the others so the optimiser moves. | Chip "at bound: omega"; the "Uncond vol (ann.)" tile turns red; narrative "unconditional vol undefined"; note "omega at bound: unconditional vol undefined". | Almost always a near-integrated fit (all variance carried by β). Use a longer history; check the closes for stale or duplicated prices. | Red: the floor is a numerical necessity. |
| α ∈ [0.0, 0.5] | The reaction to yesterday's surprise, capped at 0.5. | An α above 0.5 on daily data means one 1-σ move doubles tomorrow's variance; that is an outlier artefact, not a market. The cap keeps the solver out of that corner. | "alpha" tile red; chip "at bound: alpha"; narrative "alpha at its cap: shock response is capped by the guardrail". At α = 0 the same chip fires and the narrative says "alpha at 0: no reaction to shocks, so beta and the half-life are not identified"; the LR tile and the ν range are greyed (the boundary null assumes α interior). | At 0: GARCH adds nothing, check ΔAIC and use GBM or the bootstrap. At 0.5: look at the outlier days, try the jump model, lengthen the window. | Amber: raising the cap only matters when it binds; lowering it forces a false fit. |
| β ∈ [0.0, 0.999] | The memory of last period's variance. | β ≥ 1 is non-stationary on its own; 0.999 leaves room for α while keeping the recursion finite. | "beta" tile red; chip "at bound: beta"; narrative "near-integrated: the sample cannot pin down where vol settles; use a longer history". | Longer history; inspect the window for a regime break at its start. | Amber; never 1 or above. |
| α + β ≤ 0.9995 (STATIONARITY_MAX); flag at > 0.995 | The objective returns $-10^{10}$ above the cap; fits above 0.995 are flagged. | At α + β = 1 the unconditional variance is ω/0, the half-life is infinite and the forecast never converges; every long-run tile is meaningless. | "alpha+beta" tile red; chip "at bound: stationarity"; narrative as for β; note "near-integrated (alpha+beta > 0.995)". | Longer history. Treat the fan as valid for a few weeks only. | Amber for the flag level; the cap must stay below 1. |
| T ≥ 100 returns (_MIN_OBS) | fit_garch raises ValueError on fewer than 100 returns. | Four parameters on fewer than 100 points is noise, and the optimiser would still return something plausible-looking. | A red notice instead of a result: "Need at least 100 observations to fit GARCH; got N." | Load more history (FMP serves 504 sessions; KIS about 100 per request). | Amber upward (raise it freely); red downward. |

| Guardrail | What it is | Why it exists | What you see when it binds | What to do | Safe to change? |
|---|---|---|---|---|---|
| $\eta = 1/\nu \in [0, 0.4]$ ($\nu \geq 2.5$); NU_NORMAL = 100 | The tail parameter's box, in η. η = 0 is the normal and is not a bound hit. | The variance $\nu/(\nu-2)$ and the scaling need $\nu > 2$; below ~2.5 the likelihood is badly behaved. There is no upper bound on ν: the normal is inside the model, so the LR test is well defined (chapter 6.4). Above NU_NORMAL the t is drawn as a normal. | Lower (η at 0.4): "nu (tail)" tile red; chip "at bound: nu"; narrative "tails heavier than the model allows"; note "nu at lower bound: tails heavier than the model allows". Normal end: muted chip "normal innovations", tile "normal", narrative "normal innovations (eta = 0)" or "nu = N, effectively normal". | Normal end: the t adds nothing; read the grey LR tile and ΔAIC, a normal GARCH is enough. Lower: a few extreme days dominate; look at them, try the jump model, lengthen the window. | Amber for NU_NORMAL (a display and sampling level); red for the η ceiling (ν must stay > 2). |
| Three starts: $\beta_0 \in \{0.80, 0.90, 0.94\}$, $\alpha_0 = 0.05$, $\eta_0 = 1/8$; re-seeding from the profile scan | L-BFGS-B is run three times plus once from the normal optimum; the highest ln L wins. If a profile refit (chapter 6.5) beats it, the fit is re-run from that point. | The surface has a ridge along $\alpha + \beta \approx$ const and flat stretches in η; a single start can stop on a shoulder and report a local optimum as the fit. A start on the stationarity penalty (the old $\beta_0 = 0.95$) is dead. | Nothing when it works. A bad start shows up as converged = False or as an implausible parameter (β = 0, α at 0.5). | Nothing normally; if fits look unstable between windows, add a start rather than remove one. | Amber: adding starts is safe (slower); removing them invites local optima. |
| Observed information: Cholesky test, difference steps inside the domain | The covariance of the estimates (chapter 5.8) exists only when the information matrix is positive definite and every difference step stays inside the domain. | A singular or indefinite matrix (α = 0 leaves β unidentified; near-integrated fits next to the cap) has no meaningful inverse; printing a number there would be a lie. | Chip "SE undefined" (red) with the reason as tooltip; the tiles show the values alone; narrative "Standard errors undefined: ..."; note "standard errors undefined: ...". | Read the reason: usually the near-integrated or the alpha-at-0 case, both with their own row. The ν range (profile) still exists and is the fallback. | Red to ignore; it is a flag, not a setting. |
| Interval reaches a bound: α, β (0 or the cap), ν (2.5), persistence (1) | A 95% interval that crosses a guardrail is reported, not hidden. | A symmetric interval on a parameter near a bound is only indicative; a persistence interval reaching 1 means an integrated variance is not excluded. | Chip "interval touches a bound: alpha, beta, nu" (red); muted chip "persistence interval reaches 1"; the narrative repeats both; the half-life's upper end reads "inf". | Longer history is the only cure. The point estimates stand; the sample does not support their precision. | Amber: the rules only affect reporting. ω is deliberately left out of the chip (no tile). |
| converged (res.success of the best start) | The optimiser's own success flag. | A fit that stopped on an iteration limit or a failed line search is not a maximum; its numbers can be anything. | Chip "solver did not converge"; narrative "the solver did not converge; treat the calibration as indicative"; note "solver did not converge". | Try another window length; inspect the series for zeros, splits or duplicated closes. | Red to ignore; it is a flag, not a setting. |

| Guardrail | What it is | Why it exists | What you see when it binds | What to do | Safe to change? |
|---|---|---|---|---|---|
| at_bound detection (tolerance $10^{-4}$) | A parameter within $10^{-4}$ (relative; absolute at 0) of a bound is listed. | A bound-sitting value is the bound, not an estimate; the operator must be told. | One chip per name: "at bound: omega / alpha / beta / nu / stationarity"; the matching tile red. If nothing binds and the precision chips are clear: "GARCH guardrails clear". | Read the row for that parameter. | Amber: the tolerance only affects reporting. |
| LR_5PCT = 2.706 | The 5% point of the boundary null $½δ_0 + ½χ^2_1$ for the t-vs-normal test; also half of it is the profile interval's cut at $η = 0$. | The null $η = 0$ sits on the boundary, so half the null fits land exactly at LR = 0; a gain in 2 ln L above 2.706 occurs by chance only 5% of the time (3.841 would be 2.5%). | "LR t vs normal" tile green (above) or grey, greyed too when α sits at its bound; narrative "innovation tails heavier than normal" or "the t tail did not beat a normal GARCH on this sample"; the ν range reads "to normal" exactly when the tile is grey. | Grey: use the normal GARCH or accept that ν is uninformative on this window. | Green in effect (colour, one sentence and one interval end), but keep it the mixture's true 5% point. |

## 7.7 The modifications ladder

**Green — change freely; they change the question, not the estimator**

- **Horizon** (5 to 1,260 sessions), **paths** (200 to 100,000), **target price**: read off the same paths; more paths only shrink the Monte Carlo error (chapter 2.6).
- **Seed** (the seed argument): a different but equally valid set of paths; use two seeds to see the Monte Carlo error with your own eyes.
- **Drift mode and ERP**: move the centre of the cone; the width is untouched (chapter 8).
- **Symbol and window length** (the `days` argument of `load_history`, 504 in the app): more history makes ν and the persistence better determined and the guardrails less likely to bind; fewer than 100 returns is refused.

**Amber — allowed, but know what can go wrong**

- **NU_NORMAL** (100): the ν above which the t is drawn and displayed as normal. Raising it changes nothing visible (the excess kurtosis above 100 is under 0.07); lowering it below about 30 labels light-but-real tails as normal. It is not a bound of the fit: η runs freely down to 0. The η ceiling 0.4 (ν = 2.5) is red: the t's variance needs ν > 2, and the likelihood is badly behaved below 2.5.
- **The profile cuts** (1.92 inside, 1.353 at η = 0) and the interval-at-bound rules only decide what the ν range and the two interval chips say. The boundary cut must stay half of LR_5PCT, or the range can say "normal" while the LR tile says the t won.
- **The persistence cap** (0.9995 in the objective, 0.995 for the flag). The flag threshold is only a warning level and is safe to tune. The penalty must stay strictly below 1: at α + β = 1 the unconditional variance divides by zero and the forecast recursion no longer converges, so the uncond-vol tile, the half-life and the horizon vol all become undefined.
- **The α cap** (0.5) and the starting values. A higher α cap lets a few outliers dictate the fit; a different $\beta_0$ set is fine as long as the three starts still bracket 0.8 to 0.94 and every start satisfies $\alpha_0 + \beta_0 < 0.9995$ (a start on the penalty is dead, chapter 7.5); an $\eta_0$ far from 1/8 (say 0.38 or 0.04) starts the solver on a flat part of the surface and raises the chance of a false convergence.
- **The minimum sample** (`_MIN_OBS = 100`). Raising it is safe (the app then refuses more series); lowering it lets four parameters be fitted to a few dozen points and the numbers will be noise.
- **LR_5PCT** (2.706) decides the colour of one tile, one sentence, and the boundary end of the ν range. It is the 5% point of the boundary mixture $½\delta_0 + ½\chi^2_1$, not of $\chi^2_1$ (3.841 would be the 2.5% level); replacing it with another number makes the label "5%" a lie. Use 5.412 if you want the 1% level and say so.

**Red — a mistake here silently corrupts every number on the screen**

- **The likelihood formulas** in `_loglik` (chapter 7.3): a wrong constant, a missing ½, ν instead of ν − 2. The fit still converges; α, β, ν and every VaR are then wrong by an amount nobody can see.
- **The unit-variance scaling** $\sqrt{((\nu - 2)/\nu)}$ in `_draw_z` and the matching $(\nu - 2)$ inside the density. The two must agree.
- **The variance recursion and its start**: `_variance_path` (the filter coefficients [1, −β] and $x_t = \omega + \alpha\, \varepsilon_{t-1}^2$), its start at var(ε), the alignment that makes the last element $\sigma_{T+1}^2$, and the forecast recursion in `forecast_variance`. An off-by-one here makes the conditional vol lag or lead the data by a day and shifts every fan.
- **The ω scaling** (`_OMEGA_SCALE`) and **TRADING_DAYS = 252**: the first decides whether the optimiser moves at all, the second every annualised number in the app and the FICC pages.

## 7.8 How to tell that something broke

Two checks exist, and they catch different things. `python -m core.quant.garch` is a thirty-second self-check: it simulates a GARCH(1,1)-t series with known parameters ($\omega = 2\times10^{-6}$, $\alpha = 0.08$, $\beta = 0.90$, $\nu = 6$), refits it, and asserts α within 0.05, β within 0.06, ν in (4, 10), convergence, unit variance of the scaled draws within 3%, convergence of the 5,000-step forecast to $\omega/(1-\alpha-\beta)$ within 1%, $\ln L_t \geq \ln L_{normal}$, that every 95% interval on that series holds the truth, and that the ν range reaches the normal exactly when LR ≤ LR_5PCT. It ends by fitting a normal iid series and checking that LR ≥ 0, that the fit is effectively normal, and that it either has standard errors or says why not. If it prints "ok" the estimator is intact. The acceptance suite (`python tests/run_all.py`) is broader:

| If you broke … | This test fails |
|---|---|
| the t density, the normal density, or the recursion | test_garch_recovers_parameters_on_simulated_series (parameters drift off 0.08 / 0.90 / 6); test_garch_t_beats_normal_and_gbm_on_clustered_series (ln L ordering, LR > 2.706, ΔAIC > 0, AIC and BIC arithmetic) |
| the η parameterisation, the seeding that makes LR ≥ 0, or the profile's boundary rule | test_lr_is_nonnegative_and_correctly_sized_under_the_null (200 normal series: LR ≥ 0, 30–70% at η = 0, size within sampling error of 5%, the ν range says "normal" exactly when the LR chip does, including inside the band 2.706 to 3.841) |
| the covariance, its ω scaling, the one-sided steps, or the profile interval | test_garch_recovers_parameters_on_simulated_series (standard errors within 15% of an independent computation, every interval holds the truth); test_interval_coverage_on_simulated_t_garch (50 fits: α, β, η and the profile cover at least 85%); test_intervals_reaching_a_bound_are_flagged |
| the $\sqrt{((\nu-2)/\nu)}$ scaling in either place | test_garch_scaled_t_shocks_have_unit_variance (variance over ω must be 1, kurtosis must stay fat) |
| the recursion start, its alignment, or forecast_variance | test_garch_forecast_variance_converges_to_unconditional (first step = cond_vol_last$^2$, step 5,000 within 1% of $\omega/(1-\alpha-\beta)$, horizon vol between today and the long run) |
| a bound, the stationarity cap, the 100-observation rule, the fixed mean | test_garch_respects_bounds_and_flags_on_gbm_series |
| the drift plumbing in run_monte_carlo (which mode adds what, the $-\frac{1}{2}\sigma^2$) | test_mc_default_model_is_garch_t_seeded_and_reproducible, test_mc_zero_drift_lowers_median_on_positive_drift_series, test_drift_modes_are_ordered_and_documented, test_drift_zero_centres_the_cone_and_sample_reproduces_the_mean, test_unknown_drift_raises |
| the narrative or the numbers it quotes | test_garch_narrative_states_persistence_and_nu, test_narratives_state_their_numbers |
| a chart | test_every_chart_renders_a_data_uri_image |

A change that passes both is not proven right, but a change that fails either is proven wrong, and the failing test names the formula.

# 8. Drift

## 8.1 The least identifiable input

Volatility can be estimated from a few hundred days because every day contributes a squared surprise. The mean cannot: a sample mean over T years of a series with annual volatility σ has standard error

$$\mathrm{SE}(\hat{\mu}) = \frac{\sigma}{\sqrt{T_{\text{years}}}}$$

and nothing about sampling more often helps — daily data do not shrink it, only more years do. A 25%-vol stock over the app's two-year window has SE = 0.25/√2 = 17.7% a year, on a drift that is itself perhaps 5 to 8%. The estimate is two to three times noisier than the thing it estimates. Merton (1980) made this point about expected returns; it has

not become less true. The consequence for the fan chart is that the centre of the cone, if it is the sample mean, is essentially a random number that happens to have been positive in a bull market. The study below measured exactly how random.

## 8.2 The estimators compared

Every estimator produces an annualised log drift μ; the forecast for h sessions is $F = \mu\, h/252$ and it is scored against the realised log return $R = \ln(P_{t+h}/P_t)$. σ is the trailing 504-session annualised vol, $rf_t$ the 3-month T-bill on the origin date.

- **sample_126 / 252 / 504 / 1260**: the trailing mean of the last w daily log returns, times 252. sample_504 was the app's old default (two years).
- **zero**: $\mu = 0$. The cone centres on spot.
- **rf**: $\mu = rf_t - \sigma^2/2$. The risk-neutral log drift: a price that grows at the T-bill rate on average.
- **rf_erp**: $\mu = rf_t + ERP - \sigma^2/2$, with ERP an assumption by asset class: 4% equities and equity ETFs, 1% TLT and GLD, 0 for FX.
- **shrink**: $\mu = w \cdot \text{sample_504} + (1 - w) \cdot rf$, with $w = \tau^2/(\tau^2 + \sigma^2/2)$ and $\tau = 0.05$: the sample mean weighted by how much it can be trusted against a prior standard deviation of 5%.
- **ewma_252**: an exponentially weighted mean of all daily log returns with a half-life of 252 sessions, times 252. Long memory with a gentle emphasis on recent years.
- **capm**: $\mu = rf_t + \beta_{SPY} \times 4\% - \sigma^2/2$, with β from a 504-session regression on SPY; only for US-listed names and EWY.
- **momentum_12_1**: the mean of the last 252 sessions excluding the most recent 21, times 252, clipped to ±30%. The classic momentum signal turned into a drift.

## 8.3 The protocol

Seventeen tickers from 2005: SPY, QQQ, IWM, AAPL, MSFT, NVDA, TSLA, XOM, JPM, KO (equity), TLT, GLD (bond/gold), EURUSD, USDJPY, USDKRW (FX), EWY and Samsung Electronics 005930.KS (Korea). An origin every 21 SPY sessions; at each origin every estimator is computed from data up to that date only, and scored at h = 21, 63, 126 and 252 sessions ahead. The main pass (pass_1260) requires 1,260 prior returns and 252 forward sessions at every origin, so all methods share the same 3,135 (ticker, origin) pairs; a second pass relaxes the look-back to 504. Metrics: RMSE and MAE of F − R; bias (mean of F − R); skill = $1 - \text{MSE}/\text{MSE}_{zero}$; hit rate = share of origins with the right sign; coverage = share of realised returns inside $F \pm 1.645\, \sigma \sqrt{(h/252)}$ (target 0.90). The Diebold-Mariano test averages the squared-error differential across tickers at each origin and computes a Newey-West t-statistic on that time series (lag h/21, because origins overlap); $|t| > 1.96$ is the 5% line and a negative t means the method beats the baseline. Script: `docs/drift_study.py`; every number below is read from `docs/drift_study/results.csv`.

## 8.4 Results

### Pooled, pass_1260, h = 63

| method | n | rmse | mae | bias | skill vs zero | hit rate | coverage 90 |
|---|---|---|---|---|---|---|---|
| sample_1260 | 3135 | 0.1223 | 0.0816 | -0.0030 | +0.049 | 0.607 | 0.927 |
| ewma_252 | 3135 | 0.1247 | 0.0836 | -0.0020 | +0.012 | 0.585 | 0.920 |
| zero | 3135 | 0.1254 | 0.0850 | -0.0312 | +0.000 | | 0.927 |
| rf_erp | 3135 | 0.1266 | 0.0844 | -0.0293 | -0.018 | 0.569 | 0.928 |
| shrink | 3135 | 0.1286 | 0.0862 | -0.0341 | -0.050 | 0.491 | 0.923 |
| rf | 3135 | 0.1291 | 0.0868 | -0.0366 | -0.059 | 0.447 | 0.923 |
| momentum_12_1 | 3135 | 0.1293 | 0.0870 | -0.0095 | -0.062 | 0.549 | 0.904 |
| sample_504 | 3135 | 0.1295 | 0.0867 | -0.0018 | -0.065 | 0.566 | 0.909 |
| sample_252 | 3135 | 0.1334 | 0.0896 | -0.0001 | -0.131 | 0.556 | 0.895 |
| sample_126 | 3135 | 0.1439 | 0.0983 | -0.0013 | -0.316 | 0.542 | 0.874 |
| capm | 2002 | 0.1474 | 0.1019 | -0.0398 | -0.380 | 0.571 | 0.929 |

### Pooled, pass_1260, h = 252

| method | n | rmse | mae | bias | skill vs zero | hit rate | coverage 90 |
|---|---|---|---|---|---|---|---|
| sample_1260 | 3135 | 0.2826 | 0.1795 | -0.0183 | +0.114 | 0.667 | 0.903 |
| zero | 3135 | 0.3004 | 0.1968 | -0.1314 | +0.000 | | 0.885 |
| ewma_252 | 3135 | 0.3005 | 0.1929 | -0.0146 | -0.001 | 0.639 | 0.874 |
| rf_erp | 3135 | 0.3048 | 0.1932 | -0.1237 | -0.030 | 0.618 | 0.899 |
| shrink | 3135 | 0.3181 | 0.2049 | -0.1430 | -0.122 | 0.473 | 0.882 |
| rf | 3135 | 0.3218 | 0.2086 | -0.1529 | -0.148 | 0.419 | 0.879 |
| momentum_12_1 | 3135 | 0.3240 | 0.2141 | -0.0445 | -0.164 | 0.588 | 0.821 |
| sample_504 | 3135 | 0.3351 | 0.2172 | -0.0138 | -0.244 | 0.604 | 0.850 |
| capm | 2002 | 0.3482 | 0.2340 | -0.1655 | -0.344 | 0.641 | 0.912 |
| sample_252 | 3135 | 0.3602 | 0.2349 | -0.0067 | -0.439 | 0.590 | 0.806 |
| sample_126 | 3135 | 0.4196 | 0.2769 | -0.0117 | -0.952 | 0.567 | 0.737 |

### Diebold-Mariano, baseline = sample_504 (negative t: the method beats the old default)

| h | method | mean dloss | DM t (NW) | origins | pairs |
|---|---|---|---|---|---|
| 63 | sample_1260 | -0.00210 | -4.49 | 203 | 3135 |
| 63 | ewma_252 | -0.00132 | -4.19 | 203 | 3135 |
| 63 | zero | -0.00125 | -1.41 | 203 | 3135 |
| 63 | rf_erp | -0.00106 | -1.08 | 203 | 3135 |
| 63 | capm | -0.00103 | -0.67 | 188 | 2002 |
| 63 | shrink | -0.00052 | -0.50 | 203 | 3135 |
| 63 | rf | -0.00041 | -0.38 | 203 | 3135 |
| 63 | momentum_12_1 | +0.00022 | +0.32 | 203 | 3135 |
| 63 | sample_252 | +0.00122 | +1.59 | 203 | 3135 |
| 63 | sample_126 | +0.00387 | +3.97 | 203 | 3135 |
| 252 | sample_1260 | -0.03178 | -4.00 | 203 | 3135 |
| 252 | ewma_252 | -0.02093 | -3.90 | 203 | 3135 |
| 252 | zero | -0.02094 | -1.33 | 203 | 3135 |
| 252 | rf_erp | -0.01900 | -1.16 | 203 | 3135 |

| h | method | mean dloss | DM t (NW) | origins | pairs |
|---|---|---|---|---|---|
| 252 | capm | -0.02729 | -1.07 | 188 | 2002 |
| 252 | shrink | -0.01037 | -0.60 | 203 | 3135 |
| 252 | rf | -0.00798 | -0.45 | 203 | 3135 |
| 252 | momentum_12_1 | -0.00051 | -0.04 | 203 | 3135 |
| 252 | sample_252 | +0.02363 | +2.26 | 203 | 3135 |
| 252 | sample_126 | +0.06037 | +3.79 | 203 | 3135 |

### Diebold-Mariano, baseline = zero (negative t: the method beats a flat zero)

| h | method | mean dloss | DM t (NW) | origins | pairs |
|---|---|---|---|---|---|
| 63 | sample_1260 | -0.00085 | -1.26 | 203 | 3135 |
| 63 | ewma_252 | -0.00007 | -0.09 | 203 | 3135 |
| 63 | capm | +0.00026 | +0.96 | 188 | 2002 |
| 63 | rf_erp | +0.00020 | +1.06 | 203 | 3135 |
| 63 | sample_504 | +0.00125 | +1.41 | 203 | 3135 |
| 63 | sample_252 | +0.00247 | +2.39 | 203 | 3135 |
| 63 | momentum_12_1 | +0.00148 | +2.42 | 203 | 3135 |
| 63 | shrink | +0.00073 | +2.97 | 203 | 3135 |
| 63 | rf | +0.00085 | +3.21 | 203 | 3135 |
| 63 | sample_126 | +0.00513 | +3.89 | 203 | 3135 |
| 252 | sample_1260 | -0.01084 | -1.02 | 203 | 3135 |
| 252 | ewma_252 | +0.00001 | +0.00 | 203 | 3135 |
| 252 | capm | +0.00155 | +0.40 | 188 | 2002 |
| 252 | rf_erp | +0.00194 | +0.70 | 203 | 3135 |
| 252 | sample_504 | +0.02094 | +1.33 | 203 | 3135 |
| 252 | momentum_12_1 | +0.02043 | +2.39 | 203 | 3135 |
| 252 | shrink | +0.01057 | +2.91 | 203 | 3135 |
| 252 | sample_252 | +0.04456 | +3.04 | 203 | 3135 |
| 252 | rf | +0.01296 | +3.45 | 203 | 3135 |
| 252 | sample_126 | +0.08131 | +3.89 | 203 | 3135 |

### Per asset class, pass_1260, h = 252 (CAPM only where a same-calendar benchmark exists)

### Equity

| method | n | rmse | bias | skill vs zero | hit rate | coverage 90 |
|---|---|---|---|---|---|---|
| sample_1260 | 1814 | 0.3242 | -0.0217 | +0.152 | 0.777 | 0.926 |
| ewma_252 | 1814 | 0.3441 | -0.0143 | +0.044 | 0.745 | 0.907 |
| zero | 1814 | 0.3520 | -0.1864 | +0.000 |  | 0.894 |
| capm | 1814 | 0.3546 | -0.1766 | -0.015 | 0.647 | 0.912 |
| rf_erp | 1814 | 0.3608 | -0.1810 | -0.050 | 0.654 | 0.910 |
| momentum_12_1 | 1814 | 0.3634 | -0.0600 | -0.066 | 0.661 | 0.862 |
| shrink | 1814 | 0.3768 | -0.2066 | -0.146 | 0.483 | 0.892 |
| rf | 1814 | 0.3824 | -0.2210 | -0.180 | 0.351 | 0.881 |
| sample_504 | 1814 | 0.3892 | -0.0122 | -0.223 | 0.713 | 0.881 |

| method | n | rmse | bias | skill vs zero | hit rate | coverage 90 |
|---|---|---|---|---|---|---|
| sample_252 | 1814 | 0.4143 | -0.0001 | -0.385 | 0.660 | 0.833 |
| sample_126 | 1814 | 0.4880 | -0.0066 | -0.922 | 0.620 | 0.764 |

## Bond / gold

| method | n | rmse | bias | skill vs zero | hit rate | coverage 90 |
|---|---|---|---|---|---|---|
| shrink | 376 | 0.1566 | -0.0419 | +0.109 | 0.513 | 0.846 |
| rf_erp | 376 | 0.1566 | -0.0389 | +0.108 | 0.649 | 0.859 |
| rf | 376 | 0.1593 | -0.0489 | +0.076 | 0.574 | 0.848 |
| zero | 376 | 0.1658 | -0.0499 | +0.000 |  | 0.830 |
| sample_1260 | 376 | 0.1660 | +0.0043 | -0.002 | 0.569 | 0.851 |
| ewma_252 | 376 | 0.1680 | -0.0022 | -0.027 | 0.561 | 0.814 |
| sample_504 | 376 | 0.1827 | -0.0071 | -0.214 | 0.527 | 0.785 |
| sample_252 | 376 | 0.1829 | -0.0050 | -0.217 | 0.577 | 0.827 |
| momentum_12_1 | 376 | 0.1837 | -0.0055 | -0.227 | 0.545 | 0.816 |
| sample_126 | 376 | 0.2339 | -0.0038 | -0.990 | 0.545 | 0.710 |

## FX

| method | n | rmse | bias | skill vs zero | hit rate | coverage 90 |
|---|---|---|---|---|---|---|
| rf | 569 | 0.0817 | -0.0064 | +0.024 | 0.510 | 0.907 |
| rf_erp | 569 | 0.0817 | -0.0064 | +0.024 | 0.510 | 0.907 |
| zero | 569 | 0.0827 | -0.0140 | +0.000 |  | 0.902 |
| shrink | 569 | 0.0864 | -0.0068 | -0.092 | 0.459 | 0.895 |
| sample_1260 | 569 | 0.0941 | -0.0093 | -0.296 | 0.462 | 0.889 |
| ewma_252 | 569 | 0.0983 | -0.0082 | -0.413 | 0.460 | 0.852 |
| sample_504 | 569 | 0.1045 | -0.0081 | -0.599 | 0.427 | 0.847 |
| sample_252 | 569 | 0.1227 | -0.0069 | -1.202 | 0.487 | 0.766 |
| momentum_12_1 | 569 | 0.1258 | -0.0072 | -1.317 | 0.492 | 0.752 |
| sample_126 | 569 | 0.1520 | -0.0063 | -2.382 | 0.473 | 0.701 |

## Korea

| method | n | rmse | bias | skill vs zero | hit rate | coverage 90 |
|---|---|---|---|---|---|---|
| capm | 188 | 0.2791 | -0.0589 | +0.332 | 0.590 | 0.910 |
| rf_erp | 376 | 0.3351 | -0.1098 | +0.037 | 0.572 | 0.878 |
| zero | 376 | 0.3415 | -0.1249 | +0.000 |  | 0.872 |
| sample_1260 | 376 | 0.3437 | -0.0376 | -0.013 | 0.543 | 0.870 |
| rf | 376 | 0.3503 | -0.1498 | -0.052 | 0.452 | 0.862 |
| shrink | 376 | 0.3507 | -0.1435 | -0.055 | 0.407 | 0.854 |
| ewma_252 | 376 | 0.3725 | -0.0385 | -0.190 | 0.476 | 0.811 |
| sample_504 | 376 | 0.3942 | -0.0370 | -0.332 | 0.423 | 0.774 |
| momentum_12_1 | 376 | 0.4250 | -0.0648 | -0.549 | 0.420 | 0.731 |
| sample_252 | 376 | 0.4447 | -0.0402 | -0.696 | 0.426 | 0.715 |
| sample_126 | 376 | 0.4787 | -0.0523 | -0.965 | 0.471 | 0.694 |

**How noisy the sample drift is, per ticker (sample_504, annualised)**

| ticker | origins | mean σ | mean μ̂ | std of μ̂ | theory σ/√2 |
|---|---|---|---|---|---|
| EURUSD=X | 190 | 0.093 | -0.016 | 0.046 | 0.066 |
| USDJPY=X | 190 | 0.100 | +0.025 | 0.075 | 0.071 |
| USDKRW=X | 189 | 0.117 | +0.009 | 0.047 | 0.083 |
| TLT | 188 | 0.152 | +0.025 | 0.104 | 0.108 |
| GLD | 188 | 0.160 | +0.061 | 0.120 | 0.113 |
| KO | 188 | 0.174 | +0.090 | 0.058 | 0.123 |
| SPY | 188 | 0.178 | +0.122 | 0.077 | 0.126 |
| QQQ | 188 | 0.208 | +0.168 | 0.098 | 0.147 |
| IWM | 188 | 0.228 | +0.095 | 0.101 | 0.162 |
| XOM | 188 | 0.246 | +0.060 | 0.170 | 0.174 |
| EWY | 188 | 0.263 | +0.042 | 0.135 | 0.186 |
| MSFT | 188 | 0.264 | +0.201 | 0.117 | 0.187 |
| 005930.KS | 188 | 0.274 | +0.134 | 0.138 | 0.194 |
| AAPL | 188 | 0.287 | +0.248 | 0.161 | 0.203 |
| JPM | 188 | 0.305 | +0.137 | 0.128 | 0.216 |
| NVDA | 188 | 0.456 | +0.371 | 0.369 | 0.322 |
| TSLA | 122 | 0.552 | +0.290 | 0.471 | 0.390 |

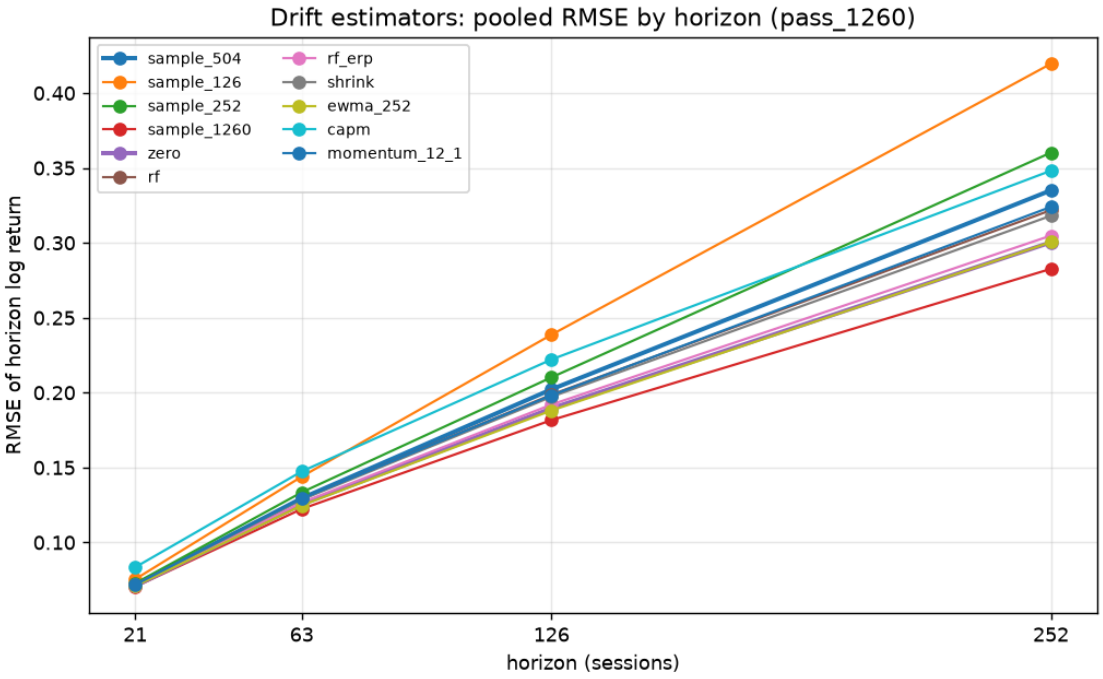



***Figure 7*** *— Drift study — pooled RMSE of the horizon log-return forecast by horizon, per estimator (pass_1260). Reproduced from docs/drift_study/fig1_rmse_by_horizon.png.*

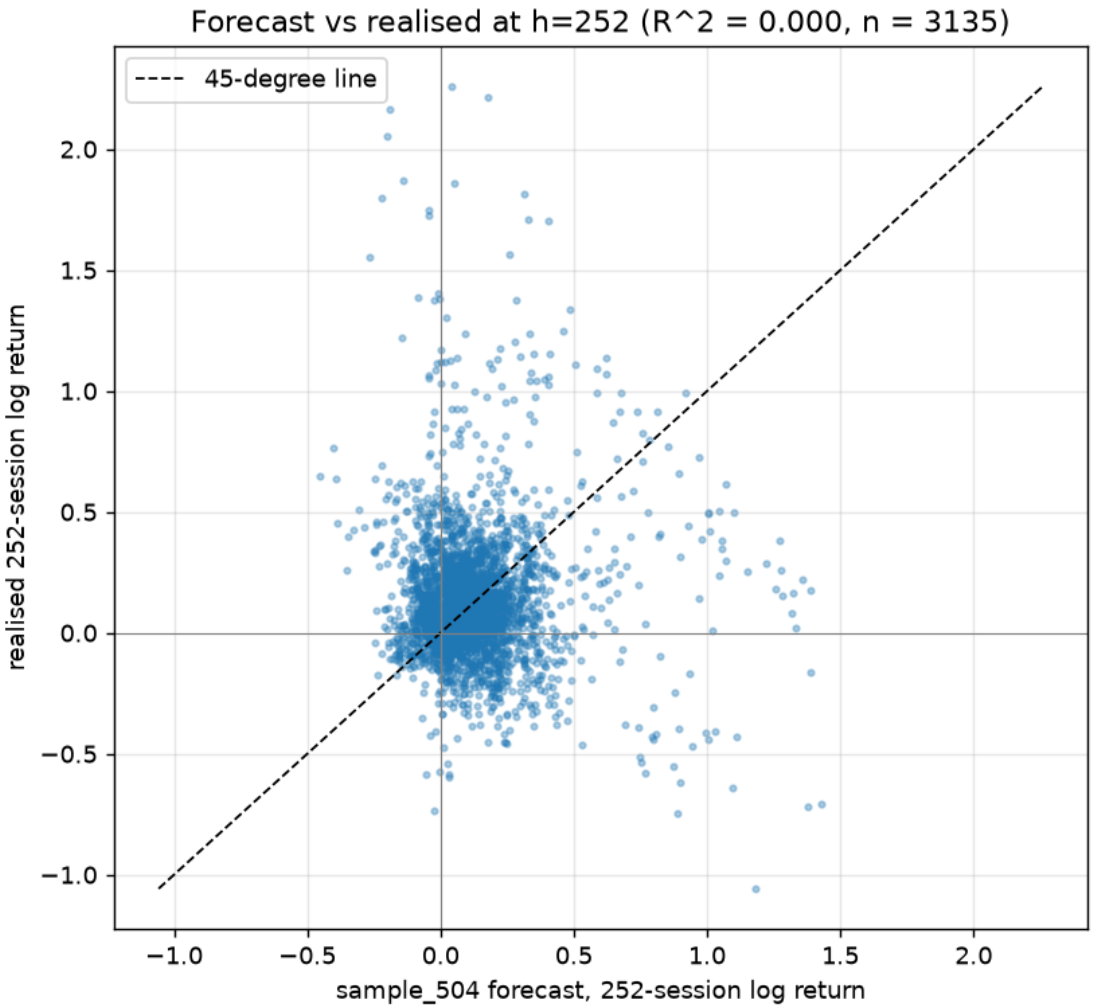



***Figure 8*** *— Drift study — the trailing 504-session sample-mean forecast against the realised 252-session log return, with the 45-degree line. The cloud has no slope: the correlation is -0.010. Reproduced from docs/drift_study/fig2_scatter_h252.png.*

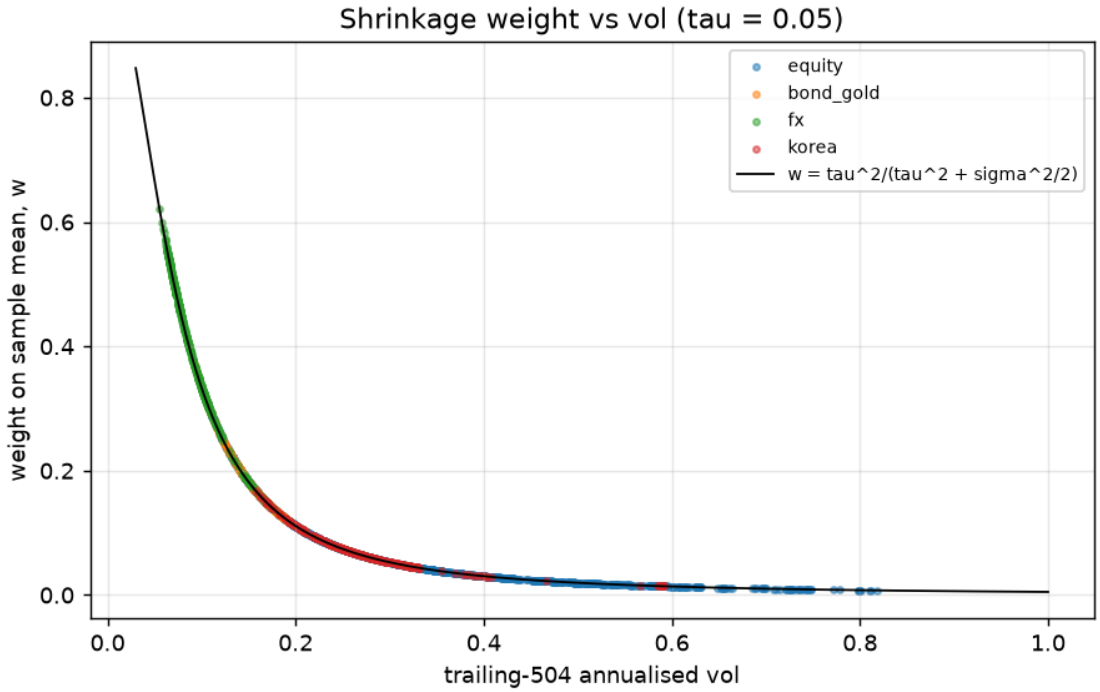



***Figure 9*** *— Drift study — the shrinkage weight w placed on the sample mean against trailing volatility (τ = 0.05). For most equities w is below 0.1, so the shrink estimator is almost pure rf. Reproduced from docs/drift_study/fig3_shrink_weight.png.*

## 8.5 Reading the results

- **The trailing two-year mean carries no forecast information.** Its correlation with the realised 12-month return is -0.010 (figure 8). Per ticker, the estimate itself swings from origin to origin with a standard deviation of the same order as the volatility: NVDA's two-year drift moved with a standard deviation of 36.9% a year.
- **The old default loses to a flat zero at every horizon.** Pooled RMSE at h = 252: sample_504 0.3351 against zero 0.3004 (skill -0.244); at h = 63, 0.1295 against 0.1254. The DM test against zero gives t = -1.33 at h = 252 in zero's favour — consistent in direction but not significant at 5%, so the honest statement is "no better than zero, probably worse". Shorter windows are significantly worse (sample_126 t = +3.89, sample_252 t = +3.04 against zero).
- **Longer memory helps, but it is a regime, not a rule.** sample_1260 has the lowest RMSE at every horizon (h = 252: 0.2826, skill +0.114; DM t = -4.00 against sample_504, -1.02 against zero — not significant). The gain comes entirely from the equity block (equity h = 252 RMSE 0.3242 vs zero 0.3520); in FX it loses to zero (0.0941 vs 0.0827) and in Korea too (0.3437 vs 0.3415). A five-year mean over 2010-2025 mostly learned that US equities went up.
- **rf + ERP ties zero and is never the worst.** h = 252 RMSE 0.3048 (skill -0.030, DM t = +0.70 against zero: a tie). It is the best full-sample method in FX (0.0817) and Korea (0.3351) and ties for first in bond/gold (0.1566 vs shrink 0.1566); it is never the worst in any class.
- **Pure rf and shrink are significantly worse than zero** (DM t = +3.45 and +2.91) while rf + ERP is not: the ERP is doing the work. Pure rf forecasts a negative drift for high-vol names because of the $-\sigma^2/2$ (a 40%-vol name gets rf − 8%), which is why its h = 252 hit rate is only 0.419. It is a pricing measure, not a forecast. Shrinkage as

specified collapses onto rf: the mean weight on the sample mean is 0.15 (equity 0.09, FX 0.36; figure 9).

- **CAPM adds nothing** over rf + ERP on its equity-only sample (equity h = 252: capm 0.3546 vs rf_erp 0.3608 vs zero 0.3520, all inside noise), and it needs a benchmark on the same calendar for every market. **Momentum has no edge**: RMSE 0.3240, hit rate 0.588, indistinguishable from the up-drift of the period.
- **Bias and coverage.** Every method is biased low at h = 252 (zero -0.1314, rf_erp -0.1237) because the period was an equity bull market; hit rates above 0.5 for positive-drift methods come from that, not from timing. Coverage of the 90% band barely depends on the drift (zero 0.885, rf_erp 0.899, sample_1260 0.903); only the noisy short windows pull it down (sample_126 0.737). The width of the cone is the vol model's job, and it does it.

## 8.6 What the app offers now, and the default

`DRIFTS = ('premium', 'zero', 'rf', 'sample')`. Every mode is an annualised *log* drift μ; every model draws zero-mean shocks and adds μ/252 to each step (`resolve_drift` in `core/quant/montecarlo.py`). c̄ in the formulas is each model's own per-step convexity correction ln E[exp(shock)], averaged over the horizon: ½$\sigma^2$ of the sample variance for GBM, that plus the jump compensator for the jump model, the block log-MGF increments for the bootstrap, and c($\sigma_t$) from each path's own conditional variance for GARCH-t (tabulated per ν and cap), so the mean price grows at the stated rate under every model. The app shows the arithmetic under the stat strip and the narrative repeats it.

| Mode | Dropdown label | μ (annualised log drift) | Inputs and where they come from |
|---|---|---|---|
| premium (default) | Risk premium: rf + ERP, less convexity (default) | rf + ERP − c̄ | rf: 3-month T-bill, FRED DGS3MO, keyless, cached; if unreachable the app assumes 3.00% and shows an error-toned chip "rf 3.00% assumed (FRED unreachable)". ERP: the "ERP %" field, default 4.0, hint "equity 4 / bonds-gold 1 / FX 0". |
| zero | Zero (stress view) | 0 | none; the cone centres on spot |
| rf | Risk-neutral: rf, less convexity | rf − c̄ | rf as above; the risk-neutral measure of chapter 1.4 — a pricing assumption, not a forecast |
| sample | Sample mean (noisy) | mean of the window's daily log returns × 252 | price history only; kept for transparency and labelled noisy |

On the worked series, GARCH-t, 126 sessions, 5,000 paths, seed 7, rf 3.00% and ERP 4.0%, the four modes give:

| Mode | μ (ann.) | Drift sentence shown in the app | Median terminal | P(loss) | VaR 95 |
|---|---|---|---|---|---|
| premium | +2.03% | risk premium: rf 3.00% + ERP 4.00% = expected (arithmetic) return 7.00%; log drift = that - convexity correction 4.97% (per step from each path's conditional variance, horizon average; t innovations bounded at ±22) = +2.03% | 339.39 | 48% | 28.6% |
| zero | +0.00% | zero: the cone centres on spot (stress view) | 335.28 | 51% | 29.2% |
| rf | -1.97% | risk-neutral: rf 3.00% - convexity correction 4.97% (per step from each path's conditional variance, horizon average; t innovations bounded at ±22) = -1.97%; a pricing measure, not a forecast | 332.67 | 52% | 30.0% |

| Mode | μ (ann.) | Drift sentence shown in the app | Median terminal | P(loss) | VaR 95 |
|---|---|---|---|---|---|
| sample | +19.37% | sample mean +19.37%: noisy, standard error about 20.2% on two years of data | 369.38 | 31% | 22.0% |

Why the risk-premium mode is the default: statistically it ties zero, so zero would be an equally defensible choice; the case for rf + ERP is that it is never the worst method in any asset class, its centre has an economic reading you can argue with (a T-bill rate you can look up plus a premium you chose), and it degrades to zero for FX where the ERP is 0. Use zero for stress tests. Use rf to see what the pricing pages assume. Use sample only to see what the window's own history says, knowing its standard error.

The sample mode's sentence states its own standard error (σ/√2 for two years) so the number never appears without its noise. Note the VaR column: the drift moves the centre by roughly (μ × horizon) and the tail numbers follow it, but the shape of the cone is identical across the four rows because the volatility model is.

# 9. Reading the Quant Lab screen

This chapter walks through one result top to bottom: the worked series, GARCH(1,1)-t, 126 sessions, 5,000 paths, the default drift with rf 3.00% and ERP 4.0%. The values are what the screen showed when this PDF was built from the same code. Tile names are the exact labels in `ui/quant_lab_page.py`.

## 9.1 The first stat strip

| Tile | Value here | Meaning | Chapter |
|---|---|---|---|
| Spot | 335.92 | the last close; every path starts here | 2.1 |
| Median terminal | 339.39 | 50th percentile of the terminal prices | 2.3 |
| 90% range | 240 – 478 | 5th to 95th percentile of the terminal prices | 2.3 |
| P(loss) | 48% | share of paths ending below spot; red above 50% | 2.5 |
| VaR 95 | 28.6% | 95th percentile of the horizon loss, fraction of spot | 2.4 |
| CVaR | 37.6% | mean loss on the worst 5% of paths | 2.4 |
| Drift (ann.) | +2.0% | the annualised log drift actually used (the mode's μ) | 8.6 |
| Vol (ann.) | 24.3% | GARCH: the next-session conditional vol; other models: the sample vol | 5.4 |
| P(target) | only with a target | share of paths at or above the target | 2.5 |

Under the strip, in muted text, the drift sentence: *risk premium: rf 3.00% + ERP 4.00% = expected (arithmetic) return 7.00%; log drift = that - convexity correction 4.97% (per step from each path's conditional variance, horizon average; t innovations bounded at ±22) = +2.03%*. It is the same string the narrative prints, produced by `resolve_drift`.

## 9.2 The chips

| Chip | Here | Meaning | Chapter |
|---|---|---|---|
| source: FMP / KIS / SYNTHETIC | source: FMP | where the closes came from; SYNTHETIC is grey and comes with a red notice above the strip | 0 |
| first date → last date | 2024-09-20 → 2026-09-24 | the calibration window | 1.3 |
| N paths | 5,000 paths | the Monte Carlo sample size | 2.6 |
| model label | GARCH(1,1)-t (default) | which shocks were drawn | 3.6 |
| drift: label | drift: Risk premium: rf + ERP, less convexity (default) | the drift mode | 8.6 |
| rf chip (premium and rf modes) | rf 3.00% assumed (FRED unreachable) or rf x.xx% (FRED DGS3MO date) | where the risk-free rate came from; error-toned when assumed | 8.6 |

## 9.3 The GARCH strip and the guardrail chips

| Tile | Value here (as the screen prints it) | Meaning | Red when | Chapter |
|---|---|---|---|---|
| alpha | 0.183 (-0.013 to 0.380) | reaction to yesterday's surprise; (95% interval) | at bound | 5.1, 5.8 |
| beta | 0.645 (0.283 to 1.007) | memory; (95% interval) | at bound | 5.1, 5.8 |
| alpha+beta | 0.8287 (0.5656 to 1.0918) | persistence; (95% interval, delta method) | > 0.995 (stationarity) | 5.2, 5.8 |
| Half-life (sessions) | 4 (1 to inf) | ln 0.5 / ln(α+β); (the persistence interval mapped through) | never (shows inf) | 5.3, 5.8 |
| nu (tail) | 3.1 (2.5 to 4.4) | t degrees of freedom, or "normal"; (95% profile interval, "normal" = includes η = 0) | at bound (2.5); grey when alpha at bound | 6.3, 6.5 |
| Cond vol (ann.) | 24.3% | next-session forecast $\sigma_{T+1}$ | never | 5.4 |
| Uncond vol (ann.) | 33.1% | √(ω/(1−α−β) × 252); no interval by design | omega at bound | 5.2, 5.8 |
| Horizon vol (ann.) | 31.3% | average forecast vol over the horizon, for the bounded innovations actually drawn | never | 5.5 |
| LR t vs normal | 85.0 | $2(\ln L_t - \ln L_{normal}) \geq 0$ | green above 2.706, grey otherwise or when alpha at bound | 6.4 |
| dAIC vs GBM | +165.8 | $AIC_{GBM} - AIC_{GARCH\text{-}t}$ | green above 0, grey otherwise | 7.4 |

Chips on this run: **interval touches a bound: alpha, beta, nu, persistence interval reaches 1**. The possible chips are "at bound: omega", "at bound: alpha", "at bound: beta", "at bound: nu", "at bound: stationarity", "solver did not converge", "SE undefined", "interval touches a bound: …" (all red), then "GARCH guardrails clear" when none of those applies, and the muted "normal innovations" and "persistence interval reaches 1". Chapter 7.6 has one row per chip. For comparison, the demo series (Data source = Synthetic demo) shows: **at bound: alpha, SE undefined, normal innovations**.

## 9.4 The three charts

- **Conditional vol chart** (GARCH only): the fitted $\sigma_t$ over the window against realised vol, as in figure 3. If the line ends well above the dashed unconditional level, the first weeks of the fan are wider than a GBM fan would be, and vice versa (chapter 4.3).
- **Fan chart**: the last 90 sessions of history, then the 5-95% and 25-75% bands and the median, with forty sample paths behind them and the target as a dashed line if given (chapter 2.3). The bands are percentiles per date, not paths.
- **Terminal histogram**: all terminal prices, spot as a solid line, the VaR 95 price as a dashed red line, the target in green (chapter 2.4). The distance between the solid and dashed lines is VaR in price terms; the mass to the left of the dashed line is the 5% whose average is CVaR.

## 9.5 The narrative and every sentence it can produce

The local narrative (`insights.narrate_monte_carlo`) is deterministic and needs no key; the Claude panel, when a key is set, receives the same numbers and adds an interpretation. On this run the local narrative read:

```
Across 5,000 simulated GARCH_T paths over 126 sessions, AAPL ends at a median of
339.39 against a spot of 335.92 (+1.0%). The middle 90% of outcomes spans 239.74 to
478.32; the simulation assigns a 48% probability of finishing below spot. Horizon
VaR(95) is 28.6% of position value, and the expected loss beyond that point (CVaR) is
37.6%. Calibration (GARCH(1,1)-t on 503 sessions): conditional vol today 24.3% against
an unconditional 33.1% and a 126-session horizon vol of 31.3%. alpha 0.183, beta
0.645, persistence 0.829 — a variance shock halves in 4 sessions. nu = 3.1: innovation
tails heavier than normal (LR 85.0 vs 2.706). 95% intervals: alpha -0.013 to 0.380,
beta 0.283 to 1.007, persistence 0.5656 to 1.0918 (half-life 1 to inf sessions); nu
2.5 to 4.4 by profile likelihood; the interval reaches a bound for alpha, beta, nu, so
it is indicative only; the persistence interval reaches 1, so an integrated variance
is not excluded. The unconditional vol carries no interval by design. dAIC vs GBM
+165.8 — GARCH is preferred. t innovations are capped at ±22: this fit expects a
larger day once per 118 years (P 0.4% within the horizon); the cap drops 4.61% of
innovation variance, so the fan's vol over this horizon runs 4.6% below the fit's own
forecast and VaR/CVaR exclude those days. Drift (risk-premium view): risk premium: rf
3.00% + ERP 4.00% = expected (arithmetic) return 7.00%; log drift = that - convexity
correction 4.97% (per step from each path's conditional variance, horizon average; t
innovations bounded at ±22) = +2.03%.
```

The sentences, in the order they can appear, and the condition for each:

| Sentence template | When | Chapter |
|---|---|---|
| Across N simulated MODEL paths over H sessions, SYM ends at a median of M against a spot of S (±x%). | always | 2.3 |
| The middle 90% of outcomes spans A to B; the simulation assigns a p% probability of finishing below spot. | always | 2.3, 2.5 |
| Horizon VaR(95) is v% of position value, and the expected loss beyond that point (CVaR) is c%. | always | 2.4 |
| Calibration (GARCH(1,1)-t on T sessions): conditional vol today x% against an unconditional y% [or an undefined unconditional level] and a H-session horizon vol of z%. | GARCH-t | 5 |
| alpha a, beta b, persistence p — a variance shock halves in n sessions [or never decays (integrated variance)]. | GARCH-t | 5.3 |
| nu = v [or normal innovations (eta = 0) / nu = N, effectively normal]: innovation tails heavier than normal [or the t tail did not beat a normal GARCH on this sample] [; alpha sits at its bound, so the LR reference distribution is only indicative] (LR l vs 2.706). | GARCH-t | 6.3, 6.4 |

| Sentence template | When | Chapter |
|---|---|---|
| 95% intervals: alpha a to b, beta c to d, persistence p to q (half-life m to n sessions) [or Standard errors undefined: reason]; nu x to y by profile likelihood [; indicative only with alpha at its bound] [; the interval reaches a bound for …, so it is indicative only] [; the persistence interval reaches 1, so an integrated variance is not excluded]. The unconditional vol carries no interval by design. | GARCH-t | 5.8, 6.5 |
| dAIC vs GBM d — GARCH is preferred. [or GBM is not beaten here.] | GARCH-t | 7.4 |
| Guardrails: the solver did not converge; treat the calibration as indicative; near-integrated: the sample cannot pin down where vol settles; use a longer history; tails heavier than the model allows; unconditional vol undefined; alpha at 0: no reaction to shocks, so beta and the half-life are not identified [or alpha at its cap: shock response is capped by the guardrail]. | one clause per flag | 7.6 |
| t innovations are capped at ±22: this fit expects a larger day once per N years (P x% within the horizon); the cap drops y% of innovation variance, so the fan's vol over this horizon runs z% below the fit's own forecast and VaR/CVaR exclude those days. [or Innovations are effectively normal (the t fit found no heavier tail), so they are drawn unbounded and no cap applies.] | GARCH-t, every drift mode | 6.2 |
| Calibration: annualized volatility x%. | GBM, jump, bootstrap | 3 |
| Drift (risk-premium view) / Drift / Drift (risk-neutral) / Drift (sample mean): . | always | 8.6 |
| Probability of reaching the T target: p%. | target given | 2.5 |
| Calibrated k jump days out of T (λ=… /yr, mean jump …). [or No >3σ daily moves in the sample — jump model degrades to GBM.] | jump model | 3.4 |
| The outcome cone is wide relative to spot — position sizing matters more than direction here. | (p95 − p05)/spot > 0.8 | 2.3 |
| The outcome cone is narrow — the calibrated volatility is low, so tail surprises would come from regime change, not from this distribution. | (p95 − p05)/spot < 0.25 | 2.3 |

# 10. Glossary and formula sheet

## 10.1 Symbols and terms

| Symbol / term | Meaning |
|---|---|
| $P_t$ | close on session t |
| $r_t$ | daily log return $\ln(P_t/P_{t-1})$ |
| μ | drift: expected daily (or, annualised, yearly) log return; $\mu_{annual} = 252\ \mu_{daily}$ |
| σ, $\sigma_t$ | volatility: standard deviation of the return; $\sigma_t$ is the conditional (time-t) value; $\sigma_{annual} = \sqrt{252}\ \sigma_{daily}$ |
| $\bar{\sigma}^2$ | unconditional variance ω/(1 − α − β) |
| $\varepsilon_t$ | the surprise $r_t - \mu$ |
| $z_t$ | innovation: mean 0, variance 1; N(0,1) or scaled $t_\nu$ |
| ω, α, β | GARCH(1,1) floor, reaction, memory |
| α + β | persistence; half-life ln 0.5 / ln(α + β) |
| ν | Student-t degrees of freedom; tails fatter as ν falls; normal as ν → ∞ |
| η = 1/ν | the fitted tail coordinate on [0, 0.4]; 0 is the normal, 0.4 is ν = 2.5 |
| κ | excess kurtosis $E[z^4] - 3$; for $t_\nu$: $6/(\nu - 4)$ |
| ln L | log-likelihood; loglik in the code |
| LR | likelihood-ratio statistic $2(\ln L_t - \ln L_{normal}) \geq 0$; null on the boundary: $\frac{1}{2}\delta_0 + \frac{1}{2}\chi^2_1$, 5% point 2.706 |
| observed information, SE | minus the second derivatives of ln L at the fit; its inverse is the covariance of the estimates and the standard errors are the square roots of its diagonal |
| Wald interval | estimate ± 1.96 SE; used for α, β, α + β (delta method) and, mapped through, the half-life |
| profile likelihood, profile interval | ln L re-maximised over (ω, α, β) at each η; the η within 1.92 of the maximum (1.353 at η = 0), mapped to ν |
| AIC, BIC | $2k - 2\ln L$; $k\ln T - 2\ln L$; lower is better; $\Delta AIC$ vs GBM = $AIC_{GBM} - AIC_{GARCH\text{-}t}$ |
| k, T | number of parameters; number of returns |
| h, H | sessions ahead; the horizon |
| $VaR_{95}$, $CVaR_{95}$ | 95th percentile of the horizon loss; mean loss beyond it |
| rf, ERP | risk-free rate (3-month T-bill, FRED DGS3MO); equity risk premium, an assumption |
| physical / risk-neutral measure | the distribution you expect (simulate risk with) / the one that prices hedgeable claims (drift = rf) |
| persistence, IGARCH | α + β; the integrated case α + β = 1 |
| L-BFGS-B | the bounded quasi-Newton optimiser used for the fit |
| at_bound, converged | the fit's list of bound-sitting parameters; the optimiser's success flag |
| DM test | Diebold-Mariano: a t-test on the difference of squared forecast errors between two methods |
| skill | $1 - MSE/MSE_{zero}$; positive beats a zero forecast |
| coverage | share of realised values inside the model's 90% band |

## 10.2 Formula sheet

$$r_t = \ln(P_t/P_{t-1}); \qquad P_T = P_0 \exp\left(\sum r_t\right)$$

$$\mu_a = 252\,\mu_d; \qquad \sigma_a = \sqrt{252}\,\sigma_d; \qquad \mathrm{SE}(\hat{\mu}) = \sigma/\sqrt{T_{\text{years}}}$$

$$\text{GBM: } d\ln S = (\mu - \sigma^2/2)\,dt + \sigma\,dW; \qquad r_t = (\mu - \sigma^2/2)\,\Delta t + \sigma\sqrt{\Delta t}\,z_t$$

$$\text{GARCH(1,1): } \sigma_t^2 = \omega + \alpha\,\varepsilon_{t-1}^2 + \beta\,\sigma_{t-1}^2;$$
$$\bar{\sigma}^2 = \omega/(1-\alpha-\beta); \qquad \text{half-life} = \ln 0.5/\ln(\alpha+\beta)$$

$$\text{Forecast: } \sigma_{T+h}^2 = \bar{\sigma}^2 + (\alpha+\beta)^{h-1}(\sigma_{T+1}^2 - \bar{\sigma}^2);$$
$$\text{horizon vol} = \sqrt{\text{mean}_h\,\sigma_{T+h}^2 \times 252}$$

$$\text{Scaled t: } z = t_\nu\sqrt{(\nu-2)/\nu}; \qquad \kappa = 6/(\nu-4)$$

$$\ln L_{\text{normal},\,t} = -\tfrac{1}{2}\ln 2\pi - \tfrac{1}{2}\ln\sigma_t^2 - \varepsilon_t^2/(2\sigma_t^2)$$

$$\ln L_{t,\,t} = \ln\Gamma((\nu+1)/2) - \ln\Gamma(\nu/2) - \tfrac{1}{2}\ln(\pi(\nu-2)) - \tfrac{1}{2}\ln\sigma_t^2$$
$$- \tfrac{\nu+1}{2}\ln\!\left(1 + \varepsilon_t^2/((\nu-2)\sigma_t^2)\right)$$

$$\text{LR} = 2(\ln L_t - \ln L_{\text{normal}}) \geq 0 \quad \text{vs} \quad \tfrac{1}{2}\delta_0 + \tfrac{1}{2}\chi_1^2 \;(5\%) = 2.706;$$
$$\text{AIC} = 2k - 2\ln L; \qquad \text{BIC} = k\ln T - 2\ln L$$

$$\text{Wald: } \hat{\theta} \pm 1.96\sqrt{[-H]_{\theta\theta}^{-1}};$$
$$\text{profile: } \left\{\,\eta : \ln L_p(\eta) \geq \ln L_{\max} - 1.92\,\right\} \;(1.353 \text{ at } \eta = 0); \qquad \nu = 1/\eta$$

$$L = 1 - P_T/\text{spot}; \qquad \text{VaR}_{95} = Q_{0.95}(L); \qquad \text{CVaR}_{95} = \text{E}[L \mid L \geq \text{VaR}_{95}]$$

$$\text{Drift modes: premium } \mu = \text{rf} + \text{ERP} - \bar{c}; \quad \text{rf } \mu = \text{rf} - \bar{c};$$
$$\text{zero } \mu = 0; \quad \text{sample } \mu = 252\,\text{mean}(r);$$
$$\bar{c} = \text{each model's per-step } \ln\text{E}[\exp(\text{shock})],\ \tfrac{1}{2}\sigma^2 \text{ for GBM}$$

$$\text{Monte Carlo error} \propto 1/\sqrt{N}$$

## 10.3 The worked fit in one line

```
GARCH(1,1)-t on 503 sessions, mu fixed at +0.077%/day: alpha=0.183±0.100
beta=0.645±0.185 persistence=0.829±0.134 nu=3.1 (2.5 to 4.4); 95% interval
reaches a bound: alpha, beta, nu; persistence interval reaches 1.
```

## 10.4 References


- Engle, R. F. (1982). Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation.
- Bollerslev, T. (1986). Generalized autoregressive conditional heteroskedasticity.
- Bollerslev, T. (1987). A conditionally heteroskedastic time series model for speculative prices and rates of return.
- Merton, R. C. (1980). On estimating the expected return on the market: an exploratory investigation.
- Hull, J. C. Options, Futures, and Other Derivatives (chapters on Wiener processes, Itô's lemma and Monte Carlo simulation).
- Tsay, R. S. Analysis of Financial Time Series (chapter on conditional heteroscedastic models).

- Diebold, F. X. and Mariano, R. S. (1995). Comparing predictive accuracy.